

TIMFuser: A multi-granular fusion framework for cyber threat intelligence

Chunyan Ma ^{a,b}, Zhengwei Jiang ^{a,b}, Kai Zhang ^a, Zhiting Ling ^{a,b}, Jun Jiang ^{a,b}, Yizhe You ^c,
Peian Yang ^{a,*}, Huamin Feng ^{b,d}

^a Institute of Information Engineering, Chinese Academy of Sciences, Beijing, 100085, China

^b School of Cyber Security, University of Chinese Academy of Sciences, Beijing, 100049, China

^c China Mobile Security Information Center, Beijing, 100053, China

^d Beijing Electronic Science and Technology Institute, Beijing, 100070, China

ARTICLE INFO

Keywords:

Cyber threat intelligence

TTP

Multi-granular fusion

Attack behavior extraction

Attack technique recognition

ABSTRACT

Cyber attack campaigns with multiple technical variants are becoming increasingly sophisticated and diverse, posing great threats to institutions and every individual. Cyber Threat Intelligence (CTI) offers a novel technical solution to transition from passive to active defense against cyber attacks. To counter these attacks, security practitioners need to condense CTIs from extensive CTI sources, primarily in the form of unstructured CTI reports. Unstructured CTI reports provide detailed threat information and describe multi-step attack behaviors, which are essential for uncovering complete attack scenarios. Nevertheless, automatic analysis of unstructured CTI reports is challenging. Furthermore, manual analysis is often limited to a few CTI sources. In this paper, we propose a multi-granular fusion framework for CTIs from massive CTI sources, comprising a comprehensive pipeline with six subtasks. Many current CTI extraction systems are limited by mining intelligence from a single source, thereby leading to challenges such as producing a fragmented view of attack campaigns and lower value density. We fuse the attack behaviors and attack techniques of the attack campaigns using innovative and improved multi-granular fusion methods and offer a comprehensive view of the attack. TIMFuser fills a critical gap in the automated analysis and fusion of multi-source CTIs, especially in the multi-granularity aspect. In our evaluation of 739 real-world CTI reports from 542 sources, experimental results demonstrate that TIMFuser can enable security analysts to obtain a complete view of real-world attack campaigns, in terms of fused attack behaviors and attack techniques.

1. Introduction

In recent years, as cyberspace attacks and defense technologies continue to develop, the national-level confrontation capabilities in cyberspace have gained prominence. Advanced Persistent Threat (APT) has emerged as a significant means of cyber confrontation between nations. An APT attack is a long-term and persistent cyber attack on a specific target employing advanced and concealed attack methods (Ren et al., 2022). According to an APT trends report by Kaspersky (GREAT, 2023), APT attack methods continue to evolve and become more sophisticated and diverse. With the growing defense challenges, there is a surge in demand for Cyber Threat Intelligence (CTI) in security analysis. As an crucial proactive defense measure, CTI can provide robust data support across all stages of security analysis by integrating it into the entire security detection cycle (Liao et al., 2016).

According to the *pyramid of pain* model proposed by Bianco (2013), CTI is categorized into six levels, ranging from file hashes and IP

addresses to domain names, network or host artifacts, attack tools, and Tactics, Techniques and Procedures (TTPs). The bottom three levels primarily focus on Indicators of Compromise (IoCs), which are generally easy to acquire. In particular, IOCs lack the ability to describe the characteristics of attackers, which are closely connected to the clues of the attack campaign. IOCs typically remain valid for a very brief period, often less than 2 days (Iklody et al., 2018). Moreover, studies indicate that attackers often change attack indicators, such as malicious file hashes and hosted domains, to evade detection (Li et al., 2019a; Jo et al., 2022; Milajerdi et al., 2019). While TTPs describe the high-level semantic context about APT campaigns, offering more value for practical APT detection (Cheng et al., 2023). Practically, most TTPs are concealed within unstructured CTI reports generated by security experts through the analysis of malware or traffic in the wild, often requiring manual extraction by security experts. Given the multi-source, heterogeneous, fragmented, and voluminous nature of

* Corresponding author.

E-mail addresses: machunyan@iie.ac.cn (C. Ma), jiangzhengwei@iie.ac.cn (Z. Jiang), zhangkai0216@iie.ac.cn (K. Zhang), lingzhiting@iie.ac.cn (Z. Ling), jiangjun860@iie.ac.cn (J. Jiang), youyizhe@chinamobile.com (Y. You), yangpeian@iie.ac.cn (P. Yang), fenghm@besti.edu.cn (H. Feng).

<https://doi.org/10.1016/j.cose.2024.104141>

Received 12 January 2024; Received in revised form 25 September 2024; Accepted 30 September 2024

Available online 4 October 2024

0167-4048/© 2024 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

cyber threat information, it presents a significant challenge for security analysts to effectively analyze and utilize CTIs. Hence, automating the analysis of TTPs hidden in the multi-source cyber threat information is crucial.

Unstructured CTI analysis reports, a crucial CTI source, contain more comprehensive threat information and describe multi-step attack behaviors, which is essential to unveil complete attack scenarios. Recent studies (Milajerdi et al., 2019; Rani et al., 2023; You et al., 2022; Alam et al., 2023) have attempted to extract TTPs from CTI reports and build attack behavior patterns, ultimately utilized for APT detection. However, current TTPs extraction systems (Milajerdi et al., 2019; Rani et al., 2023; You et al., 2022; Alam et al., 2023) are limited by the challenge of single-source intelligence mining, leading to issues like missing attack semantic context and lower value density (Li et al., 2022). To give a reality, when security analysts are tracking an attack campaign, due to time constraints, they may manually retrieve a limited number of CTI reports resulting in a restricted attack view based on their understanding of the threat descriptions in those reports. TTPs obtained under such circumstances are inherently incomplete, consequently causing a significant occurrence of false positives during the actual APT detection process (Wei et al., 2021). Conversely, multi-source CTIs enable security practitioners to have visibility into the threat landscape and find clues to threat detection not seen in isolated sources. Simultaneously, attack behaviors are closely linked to TTPs (also called attack patterns). For instance, the description of attack behavior: “APT29 has used spearphishing emails with an attachment to deliver files with exploits to initial victims” (ESET, 2022). “APT29 uses spearphishing emails” is typically associated with “T1566.001 - Phishing: Spearphishing Attachment”. Therefore, the completeness of attack behaviors is an crucial prerequisite for ensuring the quality of TTPs fusion.

Previous studies are dedicated to extracting IOC-related attack behaviors from a single source (Gao et al., 2020; Liao et al., 2016; Liu et al., 2022; Zhao et al., 2020; Zhu and Dumitras, 2018). However, non-IoC related attack behaviors are also crucial in identifying TTPs. Consider this description of attack behavior: “Crimson RAT adds Registry Run keys to establish persistence” (Dedola, 2020). The attack behavior “<Conficker RAT, adds, Registry Run keys>” corresponds to the “T1547 - Boot or Logon Autostart Execution: Registry Run Keys/Startup Folder” technique. Missing non-IoC related attack behaviors will make it difficult to detect the T1546 technique used by attackers. Mining from a single source and focusing on a single attack behavior type mining will lead to an incomplete view of TTPs.

Currently, only a few studies (Li et al., 2022; Guo et al., 2023) tackle the challenge of multi-source CTIs fusion. Li et al. (2022) propose a fusion method for attack behaviors from multiple sources at the attack technique level. However, the fusion of attack behavior is solely based on the Levenshtein distance with character features, neglecting the structural and semantic features. For example, “CVE-2021-44228 (a Log4j-related vulnerability)” and “CVE-2021-44227 (a WordPress-related vulnerability)” are fused due to their Levenshtein distance being less than the threshold. Guo et al. (2023) enhance the Levenshtein distance method to fuse attack group entities and develop a lightweight fusion framework for cybersecurity knowledge graphs. However, the mentioned studies only take character-level features into account when performing attack behaviors or attack groups fusion (limited to specific CTI types), neglecting structural and semantic features between entities.

To address these challenges, we present TIMFuser, a multi-granular CTI fusion framework that concurrently addresses attack behaviors and TTPs. One approach to CTI fusion involves supervised learning. However, as supervised learning requires a massive training corpus, the model’s performance is insufficient to adapt to the dynamic nature of MITRE’s Adversarial Tactics, Techniques, and Common Knowledge³

(ATT&CK). We implement an innovative unsupervised fusion method. Initially, we fuse attack behaviors across multiple CTI reports based on the structural and semantic features between entities at the attack technique level. Subsequently, we identify the TTPs hidden in the attack campaign reports (also known as CTI reports). Ultimately, we fuse TTPs at the attack campaign level to provide a comprehensive view of the attack campaign. TIMFuser offers security analysts with a comprehensive view of attack campaigns for existing or emerging threats, ultimately aiding in the prevention of potential attacks on their organizations. TIMFuser addresses a crucial gap in the automated analysis and fusion of multi-source CTIs, specifically in the multi-granularity aspect.

Although it sounds promising, solving the problem of multi-granularity CTI fusion faces some challenges. First, CTI reports are written in an informal natural language format, which is complex, lengthy, and contains much information unrelated to the attack campaign. Second, Currently, there is a lack of publicly available datasets for the fusion of attack campaigns and techniques. Third, attack behavior knowledge is dispersed across multiple reports. While individual reports covering only limited aspects of the attack behaviors, which are closely related to attack techniques. As a result, it is difficult to obtain a comprehensive TTP view for the attack campaign.

To the best of our knowledge, this is the first attempt to fuse CTIs across reports from the same attack campaign in a multi-granular manner. In short, our primary contributions can be summarized as follows:

- We propose TIMFuser, a multi-granular CTI fusion framework capable of automatically parsing CTI reports and fusing CTIs from multiple sources at both the attack technique and attack campaign level. Ultimately, TIMFuser yields a comprehensive TTPs view of the attack campaign. This can assist security analysts in making security decisions.
- We provide a novel unsupervised attack behaviors fusion algorithm that fully considers the structural and semantic features between entities. Compared with the mainstream methods, our algorithm enhances the F1-scores of macro, micro and pairwise by 1.9%, 2.74%, and 5.49%. These improvements indicate that TIMFuser not only achieves better performance but also enhances the reliability of the fused data, laying a solid foundation for the fusion of TTPs.
- We present a new perspective of attack techniques fusion at the attack campaign level employing set theory and similarity analysis. We demonstrate the effectiveness of the attack techniques fusion by designing multiple tests. The ability to fuse techniques at this level ensures minimal information overlap while retaining critical insights, which is essential for creating a complete TTP view.
- We evaluate the effectiveness of TIMFuser on a real-world attack campaign dataset comprising 739 reports with 542 CTI sources. Experimental results demonstrate that TIMFuser outperforms the state-of-the-art (SOTA) CTI fusion methods. We also discuss the benefit of TIMFuser through the case study.

The remainder of this paper is organized as follows: Section 2 provides a more detailed description of the background and motivation, and Section 3 introduces our multi-granular CTI fusion method. In Section 4, we present the experiment and analysis. Section 5 discusses this work and its some limitations. Section 6 provides the related work. Finally, we conclude the entire paper and propose future works in Section 7.

2. Background and motivation

This section first introduces some concepts of CTIs. Then, we discuss the issues of the automated analysis for CTI reports. Subsequently, we describe the background knowledge of the APT attack campaign, which poses the challenges for fusing the fragmented attack view. Finally, based on the background provided, we present a real-world attack campaign as a motivating example.

³ <https://attack.mitre.org/>

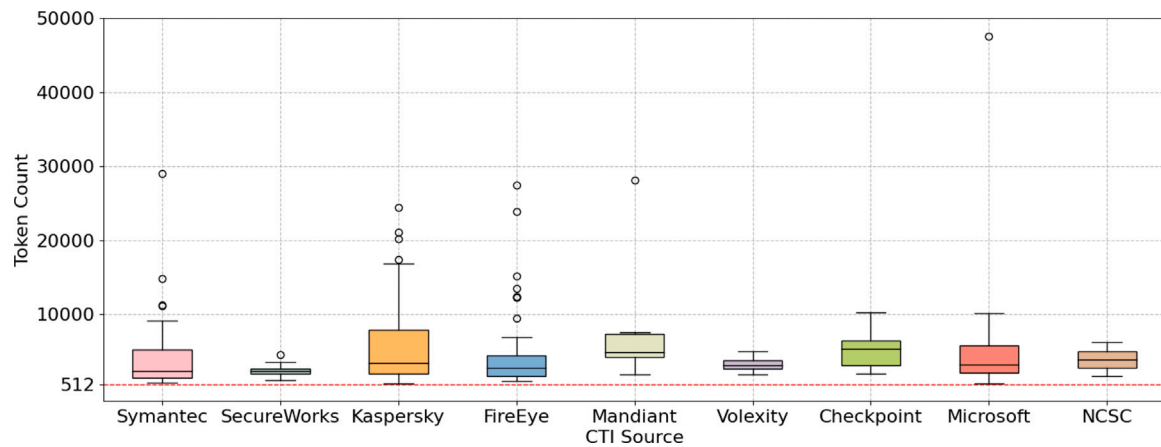


Fig. 1. The boxplot of token counts distribution. The X-axis means the CTI sources, and the Y-axis indicates the distribution of token counts. The line of box denotes the median. The bottom whisker is the minimum token count, and the top whisker indicates the maximum token count, excluding outliers marked with small circles.

2.1. Preliminaries of CTI

Currently, most of the cyber threat detection approaches (e.g., Loki (Nextron Systems, 2023a), THOR Lite (Nextron Systems, 2023b), Fenrir,⁴ etc.) rely on fragmented views of cyber threats, such as suspicious file or process names, IP addresses and domain names, to correlate suspicious events. This approach lacks accuracy in revealing how the threat unfolded, especially over long periods (weeks or even months). Indeed, the relationships between IoC artifacts are closely related to the attack behaviors (Milajerdi et al., 2019). **Attack behaviors** describe the causal dependencies between system entities (e.g., processes, files, sockets, etc., including but not limited to IoCs) that are the subjects and objects of the actions. For instance, the sentence: “APT29 used Rundll32.exe to execute payloads” can be processed by our system entity extractor to prune out the attack behavior “<APT28, use, Rundll32.exe>”. **Attack behavior graph (attack graph for short)** is a graph constructed based on the chronological order of the appearance of attack behaviors in the information flow.

However, attack behaviors contain essential clues on the attack patterns. **Attack patterns (TTPs)** depict the methods used by attackers to achieve a tactical goal and provide a high-level insight into the motivation behind an attack (Alam et al., 2023). For example, the attack behavior “<APT28, use, Rundll32.exe>” corresponds to “T1218.001 - System Binary Proxy Execution: Rundll32”. MITRE ATT&CK enumerates common attack patterns based on the analysis of the real-world attack campaign with many unique techniques for some platforms (e.g., Enterprise, Mobile, ICS).

2.2. CTI report

Many publicly available CTI reports are published in an unstructured format by the security vendors (e.g., Kaspersky, FireEye, Symantec). Common CTI reports, including APT reports and white papers, aim to disclose the details of attack campaigns or malicious software with in-depth insights and analysis. These CTI reports serve as valuable resources to help security operators make informed security decisions.

We conduct a study on the token counts of open-source CTI reports published by 10 leading security vendors, spanning the period from 2008 to 2023. Fig. 1 reveals that the majority of CTI reports from these CTI sources exceed 512 tokens, considering them as long texts. Widely used models like BERT (Devlin et al., 2018) or RoBERTa (Liu et al., 2019) are typically pretrained to process up to 512 tokens. This is problematic when feeding long reports directly into these models.

Additionally, through extensive analysis of numerous CTI reports, we observe a significant amount of irrelevant content (e.g., advertisements, team profiles). These issues become increasingly critical during the automated analysis of CTI reports. Without addressing these issues, the availability of valuable information will be affected.

2.3. Multi-stage APT attack campaign

A typical APT attack campaign comprises multiple stages and massive attack technique variants. The whole attack process is extremely stealthy, long-lasting, and tailored to specific targets. APT groups flexibly change attack techniques, gain access to the internal network through persistent network penetration, and ultimately achieve the attack goal (Stojanović et al., 2020).

Commonly, threat knowledge (e.g., malware-related) used in APT attack campaigns is published in CTI reports along with various formats, including natural language, structured, and semi-structured forms. To understand whether an organization is affected by APT attacks or is likely to be affected in the future, a security practitioner may obtain some malicious samples in the wild or analyze CTI reports to learn more about APT attack campaigns. Due to the reliance on a single intelligence source, the limitations of analytical capabilities, and the aforementioned features of APT attack campaigns, security practitioners or users obtain an extremely limited attack picture.

2.4. Motivating example

To illustrate the intuition, consider the following motivating example. Fig. 2 provides a notable supply chain attack campaign called SolarWinds compromise conducted by APT29, which has caused incalculable economic damage involving many industries (Fireeye, 2023).

After the C2 infrastructures of SolarWinds attack campaign were first seen in August 2019 (Unit42, 2020), different security vendors (e.g., FireEye, Checkpoint) and other CTI sources (formalized as $S_1, S_2, \dots, S_n$) gradually disclosed their observations in the wild and share CTIs in the form of CTI reports with the security community at various time points. For the same attack campaign, different CTI sources provide identical or complementary TTPs. For instance, S_4 (Volexity), S_5 (CrowdStrike), and S_7 (Microsoft) sources all discovered that APT29 uses PowerShell to create new tasks on remote machines, identify configuration settings, exfiltrate data, and execute other commands (“T1059.001 - Command and Scripting Interpreter: PowerShell technique”) (Cash et al., 2020; CrowdStrike, 2022; MSTIC, 2020).

⁴ <https://github.com/Neo23x0/Fenrir>

³ <https://attack.mitre.org/campaigns/C0024/>

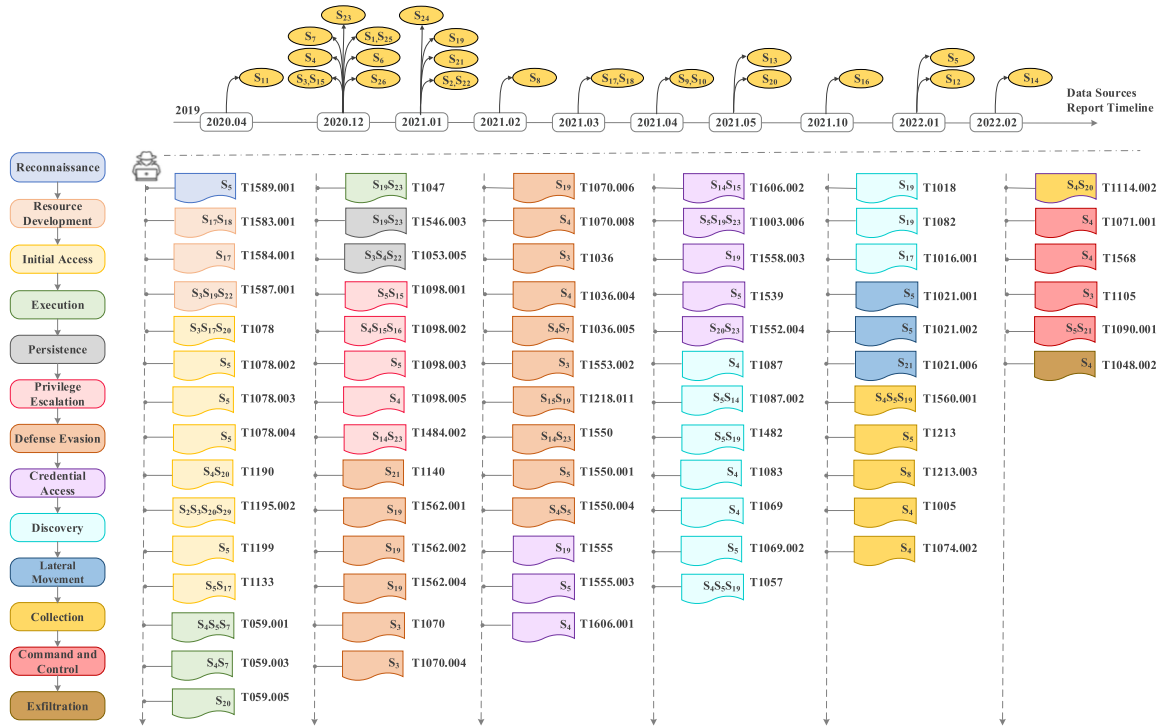


Fig. 2. A motivating example is the Solarwinds cyber attack campaign first observed in mid-December 2020.³ The top section illustrates the timeline of reports from 26 CTI sources on the SolarWinds campaign. The lower left section indicates the attack tactics represented by different colors. The lower right section denotes the attack techniques of different attack tactics disclosed by various CTI sources. Multiple CTI sources fuse to provide a complete view of the attack campaign.

However, regarding the “T1555.003 - Credentials from Web Browsers” technique, only S_5 detects that APT29 steals users’ saved passwords from Chrome (CrowdStrike, 2022). These fragmented and hidden TTPs pose a significant challenge in constructing a comprehensive view of the attack campaign.

3. Methodology

This section introduces our proposed multi-granular fusion framework shown in Fig. 3. TIMFuser consists of six components: 1) multi-source heterogeneous data collection, 2) data preprocessing, 3) relevant information identification, 4) attack graph extraction, 5) attack technique identification, and 6) attack technique fusion. *Multi-source heterogeneous data collection* is responsible for collecting data from MITRE ATT&CK and the Internet, both containing rich multi-source data. To this end, we develop two high-performance crawlers. In the *data preprocessing* section, we design the detailed preprocessing steps to address CTI text complexity challenges. *Relevant information identification* module removes CTI texts unrelated to the attack campaign and portions of text not strictly related to the attack behaviors. The *attack graph extraction* resolves the attack behaviors in CTI texts, then fuses these attack behaviors based on a novel fusion algorithm, and finally analyzes the temporal and causal order among attack behaviors (system events) for attack graph construction. The *attack technique identification* module recognizes all attack techniques related to the attack campaigns. The *attack technique fusion* is designed to fuse the attack techniques from the previous module by using set theory and similarity analysis.

3.1. Multi-source heterogeneous data collection

To our knowledge, no publicly available multi-source dataset for the same attack campaign exists. Furthermore, although open-source repositories of CTI reports are available, like APTnotes (Blanda, 2023), some reports are easily overlooked due to the different naming forms of the

report titles by the authors. This makes the data less comprehensive and complete. For this reason, we develop two different high-performance crawlers that scrape from MITRE ATT&CK and the Internet, respectively. In the crawling process, we apply several mechanisms to ensure coverage and reliability. More details are described as follows.

Since ATT&CK is usually maintained by a specialized security team, its timeliness and richness of content can be guaranteed. One crawler is responsible for scraping the procedure examples and external references of every sub-technique from ATT&CK. For another crawler, we focus on security and technology companies reporting security events to ensure quality. The crawler uses the breadth-first-search (BFS) mechanism to start from a seed URL list. Some relevant pages may contain the campaign-related keywords (the main topic of a CTI text) within the first n words of an article (where $n < 100$) (Alam et al., 2023). If the page is relevant, the crawler will save the URL. During the crawling process, these two spiders utilize selenium (Muthukadan, 2023), a headless browser automation engine, to automatically access the URLs of web pages, and then uses BeautifulSoup (Richardson, 2023) to parse the web page content rendered by selenium. The raw text format is converted by some open-source tools, such as pdfplumber⁴ and html2text.⁵ We simply clean the raw texts by removing the HTML tags and images based on a heuristic approach. Once the raw texts are cleaned, we add them to the candidate text list. Finally, we have obtained over 10748 procedure examples and 739 CTI reports as candidates (e.g., white papers, security bulletins, APT reports, security blogs, etc.).

3.2. Data preprocessing

After extracting the raw texts, we observe that they are not well-formatted. To address the CTI text complexity challenges and maximize

⁴ <https://github.com/jsvine/pdfplumber>

⁵ <https://github.com/aaronsw/html2text>

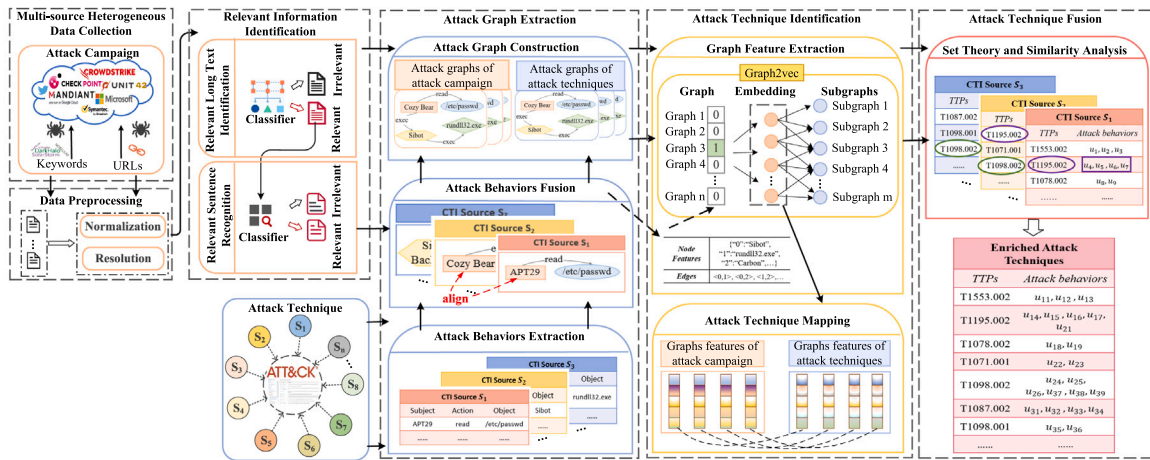


Fig. 3. The overall framework of the proposed TIMFuser.

the effect of TIMFuser, we implement the following detailed data preprocessing procedures.

3.2.1. Normalization

Normalization is mainly designed to transform sentences into a harmonized form. It consists of five subtasks: special characters removal, attack indicators reduction, passive-to-active verb conversion, and text segmentation.

Special characters removal: It is worth noting that some CTI texts may contain special characters, such as non-ASCII characters. These special characters can cause confusion and inaccuracies in the subsequent analysis. Therefore, the first step is to remove these special characters.

Attack indicators reduction: The authors of the CTI reports often rewrite certain attack indicators into another formation, considering that some users may accidentally click on the published attack indicators. For example, the malicious URL “<http://www.test.com>” is rewritten as “<http://www.test.com>”; the IP address “191.101.78.189” is rewritten as “191.101.78[.]189”. As part of the normalization process, we reduce such attack indicators.

Passive-to-active verb conversion: The passive voice can confuse the SOTA NLP (Natural Language Processing) toolkits with the subject and object of the sentence. Therefore, we convert the passive voice into the active form to facilitate the model in identifying the subject and object of the system call easily. For instance, “Emotet has been delivered by phishing emails containing attachments” be converted into “Phishing emails containing attachments have delivered Emotet”.

Text segmentation: The last step in the normalization process is text segmentation. We employ the NLTK⁶ library for both sentence segmentation and word tokenization.

3.2.2. Resolution

Unlike texts in the general domain, which follow specific grammatical rules (i.e., a sentence contains a subject–predicate–object structure at least), CTI reports contain sufficient phrases that omit the subject. We actively seek such cases. Once the subject is identified, we undertake the coreference resolution task to reconcile explicit or implicit references that pertain to the same entity into the actual object.

Ellipsis subject resolution: Initially, we use the Part Of Speech (POS) and Dependency Parsing (DP) parsing toolkits to analyze the sentence structure with an omitted subject. Upon detecting this type of sentence, we try to identify the potential subjects based on the distance from the current sentence to the subject of the neighboring sentences.

As illustrated in Fig. 4, we resolve the subject of the elliptical sentence as “UNC2452”.

Coreference resolution: We identify two types of coreference resolution in CTI reports.

- (1) **Explicit coreference:** Explicit coreference often employs pronouns in place of the original entity to avoid repetition. For instance, in Fig. 4, pronoun “it” corresponds to the entity “UNC2452”. For explicit coreference resolution, we utilize the NeuralCoref⁷ model.
- (2) **Implicit coreference:** In contrast to explicit coreference, implicit coreference resolution tends to use certain words or phrases to substitute the previous entity. To ensure the broad comprehensibility of CTI reports and meet the needs of different readers, CTI reports are written in a highly diverse style. Ambiguous synonyms may appear in different reports. For example, C2, C&C, Command and Control are different representations but refer to the same entity. Additionally, some verbs like clone and spawn represent an action about a fork system call. The dependencies among such entities cannot be recognized with common NLP models. Therefore, we construct mapping dictionaries based on domain knowledge, aiming to map different synonyms of nouns and verbs present in CTI reports to entities and verbs that may be observed in the system audit logs. We construct dictionaries of synonyms mapping for entities and verbs in Tables 1 and 2 respectively. Given that the CAR (Cyber Analytics Repository) model (MITRE, 2023) of MITRE provides high-level modeling and analysis, we consider the potential threat actions in the CAR model shown in Table 3, such as email and file. For other types of verbs, we define a verb mapping dictionary through a comprehensive analysis of real-world CTI reports.

3.3. Relevant information identification

After preprocessing, we identify a considerable number of lengthy reports that are unrelated to attack campaigns. Moreover, some relevant CTI reports contain substantial statements (e.g. advertisements) unrelated to attack details. In this section, we initially identify long reports as security-related or non-security-related. Then, we focus on extracting sentences that are pertinent to attack campaigns. This process is pivotal in simplifying the complexity and enhancing their overall quality.

⁶ <https://github.com/nltk>

⁷ <https://github.com/huggingface/neuralcoref>

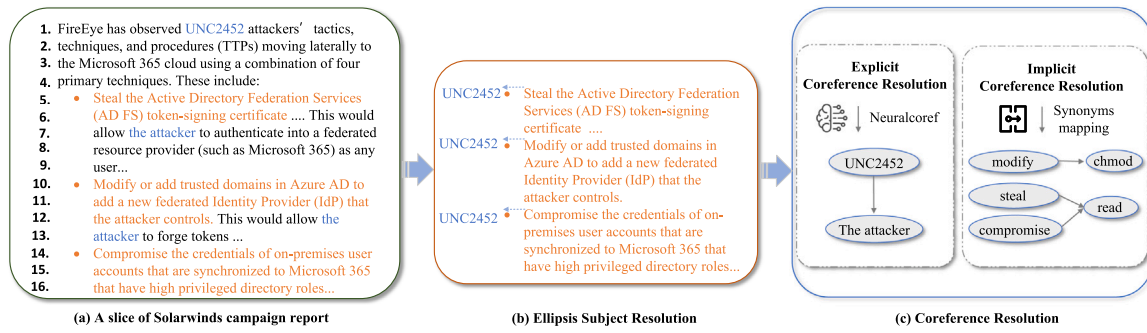


Fig. 4. Resolution steps to turn a CTI report into a digestible form with ellipsis subject resolution and coreference resolution.

Table 1
Mapping dictionary (part) for entity synonyms.

Entity	Synonyms
C&C	C&C, C2, Command and Control, CC, CnC, CandC, ...
%System32%	%SYSTEM32%, <SYSTEM32>, SYSTEM32_, % SYSTEM32%, ...
%TEMP%	%TEMP%, <TEMP>, TEMP_, % TEMP %, < TEMP >, ...
%WINDOWS%	%WINDOWS%, <WINDOWS>, WINDOWS_, ...
...	...

Table 2
Mapping Dictionary For Verb Synonyms.

Verb	Synonyms
Write	write, form, entrench, place, exfiltrate, deploy, implant, drop, install, putfile, compose, create, copy, save, add, modify, append, timestamp, edit
Read	survey, read, gather, download, navigate, locate, get, acquire, check, detect, record, exfiltrate, extract, obtain, access
Unlink	unlink, delete, clear, remove, erase, wipe, purge, expunge
Send	send, transfer, post, postsinformation, move, transmit, deliver, push, redirect
Receive	receive, accept, take, get, collect
Connect	connect, click, browse, portscan, bind, listen, communicate
Fork	fork, clone, spawn, issue, set
EXEC	use, execute, executed, run, launch, call, perform, list, invoke, inject, open, target, resume
Exit	exit, terminate, stop, end, finish, break off, abort, conclude, block, quarantine, close
MMAP	allocate, assign

3.3.1. Relevant long text identification

We consider the relevant long text identification task as a binary classification problem, where texts are categorized as either *security-related* or *non-security-related*. Existing classification approaches for CTIs partition or shorten the long text into smaller sequences that fall within the typical 512-token limit of BERT-style pretrained models. For instance, some methods focus on the beginning or ending sections of a report, but this method is incapable of perceiving contextual information from the entire sequence. This limitation arises because the beginning or concluding parts of CTI reports usually contain generalized background descriptions or irrelevant content. Consequently, such

Table 3
CAR data model provided by ATT&CK.

Object	Action
email	block, delete, deliver, redirect, quarantine
file	acl_modify, create, delete, modify, read, timestamp, write
process	access, create, terminate
registry	add, key_edit, remove, value_edit
socket	bind, close, listen

methods may lead the model to make erroneous judgments due to the loss of crucial information.

To mitigate the above problem, we employ Longformer (Beltagy et al., 2020) to build the contextual representation for long CTI reports. Longformer utilizes the efficient self-attention that scales linearly with the length of the input sequence. Unlike the typical self-attention component, Longformer introduces the sparsity to the full self-attention matrix through an “attention pattern”, specifying pairs of input locations attending to one another. This approach combines a windowed local-context self-attention with end task-motivated global attention, encoding inductive bias about the task. The local attention contributes to building the contextual representations, while the global attention enables our model to construct full sequence representations. To train our model, we manually label a balanced dataset.

3.3.2. Relevant sentence recognition

To reduce the verbosity and capture the attack details in CTI reports, we introduce the relevant sentence recognition task to distinguish the sentences containing attack details. To do this, we need to construct a fine-grained representation of CTI text. The relevant sentence recognition model is shown in Fig. 5. Currently, one of the best fine-grained representation models BERT is leveraged for contextual CTI text. To address the long-distance dependencies, we incorporate the BiLSTM (Bi-directional Long Short Term Memory) layer for contextual information extraction. Relevant text usually implies multiple concerns such as attack means, target, time, etc. We design a multi-head self-attention mechanism aimed at understanding the different types of hidden contextual information. Finally, a linear layer and softmax layer are added for the final prediction.

3.4. Attack graph extraction

In the previous step, we recognized the relevant sentences from the relevant reports. The irrelevant sentences will be removed. Subsequently, TIMFuser extracts the attack graph for every relevant report using the semantic role labeling (SRL) method and a predefined set of rules. In an ideal attack graph, nodes correspond to subjects and objects, while edges represent the verbs.

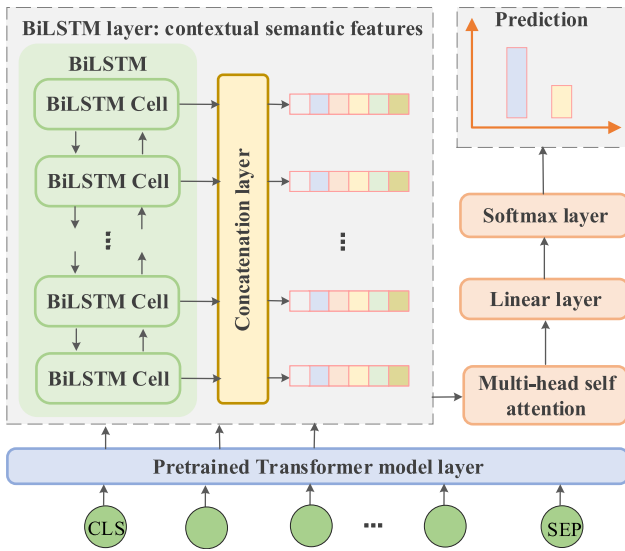


Fig. 5. Relevant sentence recognition model.

Table 4
Semantic role arguments and associated labels used in SRL (Palmer et al., 2005).

Label	Argument
ARG0	Agent
ARG1	Patient
ARG2	Instrument, Benefactive, Attribute
ARG3	Starting point, Benefactive, Attribute
ARG4	Ending point
ARGM	Modifier

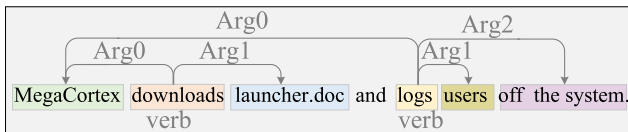


Fig. 6. Semantic roles parsed by SRL. Words on the arch present the labels.

3.4.1. Semantic role labeling

SRL is an NLP task that aims to analyze the predicate-argument structure within sentences by determining the semantic roles (called *argument*) played by each word or phrase given a specific predicate. SRL is typically designed to answer questions about basic event structures such as “*who did what to whom when where and why*”. In essence, the primary goal of SRL is to identify all component words filling semantic roles for a predicate verb and then assign the semantic labels for each word or phrase in a sentence. Table 4 illustrates the correspondence between labels and arguments. To provide an intuitive understanding of SRL, consider the example in Fig. 6. In this sentence, two verbs, *downloads* and *logs*, are present. The labels Arg0, Arg1 and Arg2 denote the *subject*, *object*, and *indirect object* of the respective verb. SRL detects the possible arguments related to a verb and yields the final results: “(Arg0: *MegaCortex*, verb: *downloads*, Arg1: *launcher.doc*)”, “(Arg0: *MegaCortex*, verb: *logs*, Arg1: *users*, Arg2: *off the system*)”.

To do this, we use a publicly available model (He et al., 2017). This model incorporates highway bidirectional LSTMs with constrained decoding and has been trained on a large corpus. Note that we do not retrain the model further, instead of using the output of the model directly.

3.4.2. SRL post-processing

- (1) **System entities detection:** After SRL processing, a substantial number of component words for filling the roles may be

obtained. Upon inspection, we find that some roles are not system entities (e.g., process, file, socket). For instance, “(Arg0: *MegaCortex*, verb: *logs*, Arg1: *users*, Arg2: *off the system*)” obtained in Fig. 6 does not include the system entities. Consequently, we devise specific rules for detecting potential system entities within semantic roles. We eventually **prune away** those that are not system entities.

- (2) **System actions transformation:** Next, TIMFuser parses the verbs resulting from SRL. This is because some verbs are not the system call actions (e.g., fork, clone, write). Based on the verb synonyms mapping dictionary in Section 3.2.2, TIMFuser transforms the verbs into the system call actions. For example, *download* in Fig. 6 is transformed into *read*. If a verb is not a system call action, TIMFuser will **prune away** it.
- (3) **Causal inference:** After the previous steps, we can get massive “(Arg0, verb, Arg1)” tuples, which are considered as “(subject, action, object)” (also called *attack behaviors*). In this case, we are unable to know the direction of the information flow between subjects and objects. TIMFuser uses a mapping of system calls to infer the direction of system flows in the same way as Satvat et al. (2021). The mapping associates each system call with the direction of the information flow. For instance, the *fork* system call implies the flow from *subject* to *object*, while the *receive* system call indicates flow direction from *object* to *subject*.

3.4.3. Attack behaviors fusion

In this step, TIMFuser performs fusion at the attack technique level. The whole process involves the fusion of attack behaviors across multiple CTI sources. Several sources are used when constructing an attack graph, which possibly prompts entity (includes subject, object, action) disambiguation and duplication (both called *canonicalization*) ignored in previous steps. As a result, it is essential to apply the fusion technique to address these matters.

To further clarify the importance of addressing these issues, consider the following two triple extractions: “<APT29, exec, cmd.exe>”, “<UNC2452, read, /etc/passwd>”. Without considering ambiguity and redundancy, these extractions might be treated separately, leading to isolated nodes without connecting edges. However, “APT29” and “UNC2452” are perceived as the same entity. Another case involves entity canonicalization, such as “*Sibot backdoor*” versus “*Sibot*”, and “*Equation group*” versus “*Equation*”. Such attack graphs will suffer from redundant facts. Previous entity canonicalization methods (Piplai et al., 2020; Li et al., 2022; Guo et al., 2023) ignore such available side information and perform canonicalization in isolation focusing only on the triples (e.g., based on edit distance). We use the contextual embedding and side information of entities to tackle this problem. The potential of using contextual embedding to address the canonicalization problem has been demonstrated by Sarhan and Spruit (2021), Vashishth et al. (2018).

To be specific, we employ the fusion architecture (see Fig. 7) for the attack technique level fusion process. The architecture comprises the following key components:

- (1) **Acquiring Side Information:** We use the following side information to get equivalent entities and actions.
 - (a) **Morphological Normalization (M-norm):** A variety of morphological normalizations are used to recognize the same entities, including tense recognition, pluralization, capitalization, and other common features in natural language descriptions.
 - (b) **Entity Linking:** Given a subject and object, we use the entity linker implemented by spaCy⁸ to link them (i.e. entity mentions) to the knowledge base such as Wikipedia,

⁸ <https://spacy.io/>

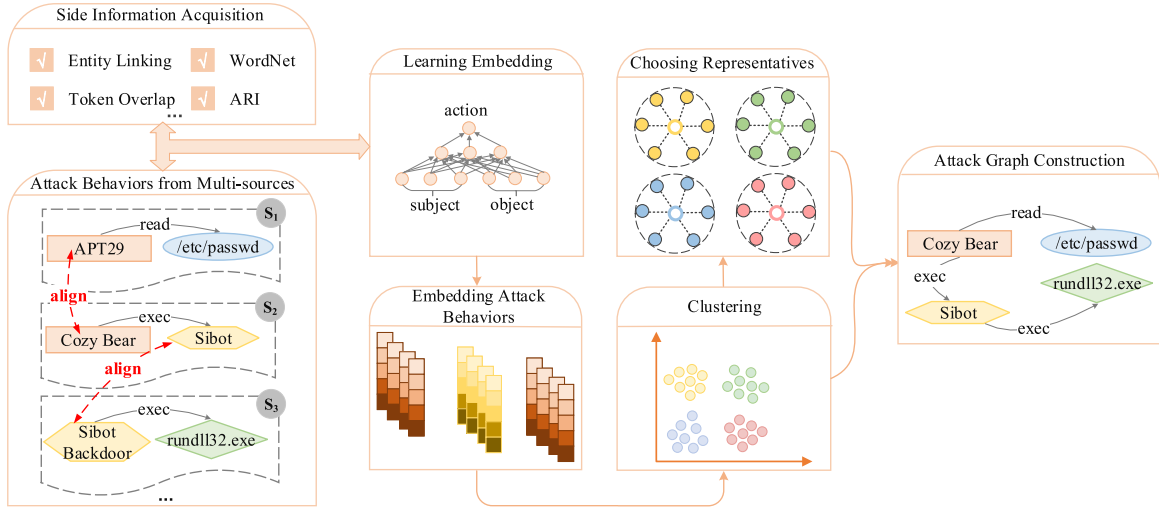


Fig. 7. Overview of the proposed attack behavior fusion method.

etc. Through entity linking, “APT29” and “UNC2452” are linked to the same entity “Cozy Bear”. Such entity mentions are potentially considered equivalent.

- (c) **WordNet with Word-sense Disambiguation:** Wordnet (Miller, 1995) provides the possible synsets for a given subject or object. For example, “variant” and “edition” link to the same synset *version.n.02* by using wordnet with word-sense disambiguation. If two entity mentions share the same synset, they have strong evidence to be deemed as the same entity.
- (d) **IDF Token Overlap:** This criterion measures the degree of overlap of entity mentions. For example, “Cardinal RAT” and “Cardinal” are more likely to be recognized as the same entity. The degree of IDF token overlap can be defined as:

$$s_{idf}(m, m') = \frac{\sum_{t \in w(m) \cap w(m')} \log(1 + f(t))^{-1}}{\sum_{t \in w(m) \cup w(m')} \log(1 + f(t))^{-1}} \quad (1)$$

where $w(\cdot)$ denotes the terms set for a subject or object, excluding stop words. $f(\cdot)$ is the document frequency of a token from the terms set $w(\cdot)$.

- (e) **Association Rule Information (ARI):** We use the statistic rule to infer the equal actions. Specifically, two actions r and r' are equivalent if and only if both $r \Rightarrow r'$ and $r' \Rightarrow r$ satisfy the support and confidence thresholds.

- (2) **Learning Embeddings:** Some of the equivalent relations obtained from different side information may be spurious, so we set the *penalizing factor* for each side information as $\lambda_{sub/obj/act, \theta}$. We optimize the objective function of CESI (Vashishth et al., 2018) chosen for its effectiveness in tasks like entity canonicalization in knowledge graph construction. Given that we only have positive attack behaviors, we generate negative examples using heuristics such as the local closed world assumption. To rank the probability of positive triples higher than the probability of negative ones, we employ a pairwise ranking loss. Additionally, we incorporate the reliability (by using a penalizing factor) of

side information into the final objective function as follows.

$$\begin{aligned} \min_{\theta} \lambda_{str} \sum_{i \in X_p} \sum_{j \in X_n} \max(0, \gamma + \sigma(\eta_j) - \sigma(\eta_i)) \\ + \sum_{\theta \in \ell_{sub}} \frac{\lambda_{sub, \theta}}{|\Phi_{sub, \theta}|} \sum_{s, s' \in \Phi_{sub, \theta}} \|e_s - e'_s\|^2 \\ + \sum_{\theta \in \ell_{obj}} \frac{\lambda_{obj, \theta}}{|\Phi_{obj, \theta}|} \sum_{o, o' \in \Phi_{obj, \theta}} \|e_o - e'_o\|^2 \\ + \sum_{\phi \in \ell_{act}} \frac{\lambda_{act, \phi}}{|\Phi_{act, \phi}|} \sum_{a, a' \in \Phi_{act, \phi}} \|e_a - e'_a\|^2 \\ + \lambda_{reg} \left(\sum_{s \in subs} \|e_s\|^2 + \sum_{o \in objs} \|e_o\|^2 + \sum_{a \in acts} \|e_a\|^2 \right). \end{aligned} \quad (2)$$

$$\sigma(x) = \frac{1}{1 + e^{-x}} \quad (3)$$

$$\eta_i = e_a^T (e_s \star e_o) \quad (4)$$

Noting that, $\Theta = \{e_s\}_{s \in subs} \cup \{e_o\}_{o \in objs} \cup \{e_a\}_{a \in acts}$, denotes the set of all *subjects*, *objects* and *actions* d-dimensional embeddings, where *subs*, *objs* and *acts* are the set of all *subjects*, *objects* and *actions* in the input. X_p , X_n refer to the positive and negative examples and $\gamma > 0$ specify the width of the margin. ℓ_{sub} , ℓ_{obj} , ℓ_{act} are the set of all types of *subject*, *object*, and *action* side information. For instance, $\ell_{sub} = \{\text{Entity Linking, WordNet, } \dots\}$, $\ell_{act} = \{\text{ARI}\}$. $\Phi_{sub, \theta}$, $\Phi_{obj, \theta}$, $\Phi_{act, \theta}$ specify the equivalence conditions of subject, object, and action from the side information θ or ϕ .

To combine the expressive power of the tensor product with the efficiency and simplicity of TransE (Bordes et al., 2013), we use the circular correlation of vectors to represent pairs of entities as Eqs. (4) and (5), where $\star : R^d \times R^d \rightarrow R^d$. In the last term, we add the L2 regularization term to avoid overfitting.

$$[a \star b]_k = \sum_{i=0}^{d-1} a_i b_{k+i \bmod d} \quad (5)$$

- (3) **Clustering Embeddings:** After obtaining the embeddings learned in the previous step, we use the Hierarchical Agglomerative Clustering (HAC) (Kobren et al., 2017) algorithm to identify similar *subjects*, *objects*, and *actions* in space based on the distance metric of cosine similarity. For example, a cluster comprises “APT29”, “Cozy Bear”, “UNC2452”, etc. Our choice behind HAC is motivated by the fact that it does not require prior knowledge regarding the number of clusters and it can partition clusters

based on semantic hierarchies, where each cluster corresponds to a set of semantically similar *subjects*, *objects*, and *actions*. Additionally, it supports complete linkage clustering, similar to the concept of farthest neighbor clustering. Initially, each average embedding is a cluster on its own, and in each step, the two clusters having the smallest maximum pairwise distance are merged. Compared to single and average linkage criterion, complete linkage criterion can give smaller sized clusters. This is more reasonable for our canonicalization problem.

- (4) **Choosing Representative:** In the next phase, we determine a representative for each *subject*, *object*, and *action* cluster. For each cluster, we calculate the mean of all elements' embeddings weighted by the number of occurrences of each element in the input. The *subject*, *object*, and *action* that lie closest to the weighted cluster mean is selected as a representative of the cluster. We formally define the equations for choosing a representative in the cluster C as Eqs. (6) and (7), where C denotes any one cluster in the *subjects*, *objects*, or *actions* clusters.

$$Mean(C) = \frac{\sum_{x \in C} e_x \cdot f(x)}{\sum_{x \in C} f(x)} \quad (6)$$

$$Rep(C) = \arg \min_{x \in C} \|e_x - Mean(C)\| \quad (7)$$

where $Mean(\cdot)$ denotes the weighted average embedding of a cluster. $f(\cdot)$ is the document frequency of the *subject*, *object*, or *action*, and $Rep(\cdot)$ refers to the representative of the cluster.

- (5) **Constructing Attack Graphs:** In the final step, we build the attack graph, which can fuse multi-source attack behaviors together based on their correlations. This allows us to observe and analyze the attack behaviors of malware or actors from a global perspective. For each directed information flow between *subject* and *object*, we generate edge and node pairs (e.g., $APT29 \xrightarrow{exec} cmd.exe$) as subgraphs at the sentence level. Currently, we only consider dependencies within a single source. Multi-source dependencies will be established based on semantically similar nodes, i.e., the clusters obtained in the previous steps. We canonicalize each node and edge (*action*) using the representatives of clusters which include these nodes and edges. Naturally, we merge the similar nodes to fuse the subgraphs into a complete attack graph.

In summary, the whole procedure of fusing attack behaviors is presented in Algorithm 1. For each technique, including the sub-techniques from MITRE ATT&CK, we construct the attack graphs denoted as G_q by using our fusion algorithm. To comprehend the TTPs view of an attack campaign, we also build the attack graphs symbolized as G_p for all CTI reports related to the same attack campaign.

3.5. Attack technique identification

The primary goal of attack technique identification is to identify the TTPs associated with the attack campaign. Some prior research (Li et al., 2022) construct attack technique templates based on the ontology model. However, this approach has limitations in terms of feature richness and flexibility. Other studies (Alam et al., 2023; Abdeen et al., 2023; Grigorescu et al., 2022) utilize embedding models, capable of capturing the semantic similarity of attack behavior elements but lacking in structural information. In this section, we adopt the graph embedding approach to match G_q with G_p for TTPs identification.

We model the attack technique recognition task as the subgraph matching problem. So we need to represent the graph. Inspired by the Graph2vec (Narayanan et al., 2017) method, we depict the attack graph as a document, illustrated in Fig. 8, where nodes correspond to words, subgraphs represent sentence or paragraph sequences. For graphs G_p and G_q , we employ Graph2vec, one of the best performing graph

Algorithm 1: Attack behaviors fusion

Input: Dataset: Attack behaviors dataset (X_p); Initialized model parameters; Set the hyperparameters: the number of epoch for training ($epochs$), number of batch ($batch_size$), distance threshold of HAC clustering (d_t).

Output: Trained embeddings for attack behaviors; Fused attack behaviors.

```

1  $S \leftarrow \emptyset$  // Initialize side information set
2 for  $x$  in  $X_p$  do
3   Acquire side information  $S_i$  for a  $x$ ;
4    $S = S \cup S_i$ ;
5 end
6 Initialize embedding for  $X$ ;
7 for  $e=1$  to  $epochs$  do
8   Shuffle  $X$ ;
9   for mini-batch  $b$  in  $batch\_size$  do
10    Generate the negative samples  $X_n$  from  $X$ ;
11    Merge all the datasets:  $X_t = X_n \cup X_b$ ;
12    Given  $X_t$  and  $S$ , compute gradient by using Eq. (2);
13    Update the parameters of model;
14   end
15 end
16 Get the learned embeddings  $E$ ;
17  $d_m \leftarrow cosine$ ; // Distance metric
18  $l_c \leftarrow complete\_linkage$ ; // Linkage criterion
19  $clusters = HAC(E, d_m, l_c, d_t)$ ;
20  $clusters_{rep} \leftarrow \emptyset$ ;
21 for  $c$  in  $clusters$  do
22   Get the average embedding of all elements in  $c$  by using Eq. (6);
23   Choose the representative ( $c_r$ ) of  $c$  based on Eq. (7);
24    $clusters_{rep} = clusters_{rep} \cup c_r$ ;
25 end
26  $Construct\_Subgraphs(X_p)$ ;
27  $Canonicalize(clusters_{rep}, clusters, X_p)$ ;
28  $FuseSubgraphs(X_p, clusters)$ .
```

embedding methods. Graph2vec, an unsupervised approach, transforms graphs into vector representations using a doc2vec-based technique. It is trained to maximize the likelihood of predicting subgraphs within the input graph, and eventually, graphs with similar subgraphs and similar structures have similar embeddings. We feed our constructed attack graph dataset and node features (an output of Algorithm 1) into the Graph2vec model and obtain vector representations from the hidden layers. Subsequently, with z_p and z_q as the embeddings of G_p and G_q , we use $D(z_p, z_q)$ to quantify the distance of G_p and G_q following Eq. (8).

$$D(z_q, z_p) = \frac{z_q \cdot z_p}{\|z_q\| \|z_p\|} \quad (8)$$

The graph matching function Eq. (9) is used to measure the matching degree of subgraphs based on a defined threshold t . If $f(z_p, z_q) = 1$ exists, then G_q is a subgraph of G_p . Otherwise, they do not conform to subgraph constraints.

$$f(z_q, z_p) = \begin{cases} 1 & \text{if } D(z_q, z_p) < t \\ 0 & \text{otherwise} \end{cases} \quad (9)$$

3.6. Attack technique fusion

Unlike the attack behavior, attack technique is a high-level CTI. Accordingly, we introduce a novel approach for attack technique fusion at the level of the attack campaign. Inspired by Azevedo et al. (2019),

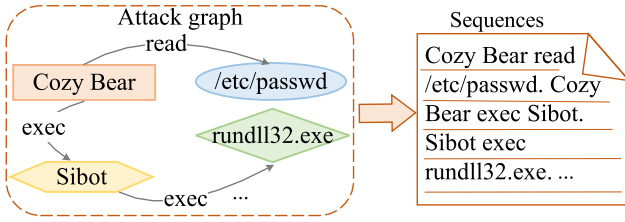


Fig. 8. Attack graph can be processed into a document.

which aggregate and correlate different IoCs from various sources to generate enriched IoCs, we aggregate all attack techniques using *set theory* and similarity measures analysis. That means an attack technique can be conceptualized as a set whose elements are its attributes. The attributes are composed of tuples, like “<attack tactic, technique name>”. For a CTI source S_i , its disclosed attack techniques are: $ST_i = \{ \langle a_{i,1}, t_{i,1} \rangle, \langle a_{i,2}, t_{i,2} \rangle, \dots, \langle a_{i,n}, t_{i,n} \rangle \}$, $ST_i \subset ST$. To gain insights into the details of attack techniques, we record the attack behavior lists (i.e., *usage*) corresponding to attack techniques as $SU_i = \{ u_{i,1}, u_{i,2}, \dots, u_{i,n} \}$, $SU_i \subset SU$, where $u_{i,1}$ refers to the usage list of $\langle a_{i,1}, t_{i,1} \rangle$. During the process of attack techniques fusion, any two different CTI sources, S_i and S_j , may exist in the following cases:

- Case 1: ($ST_i \cap ST_j = \emptyset$), ST_i and ST_j are unrelated;
- Case 2: ($(ST_i \cap ST_j \neq \emptyset) \wedge (ST_i \not\subset ST_j \wedge ST_j \not\subset ST_i)$), ST_i and ST_j are related and two sets have complementary elements, in which case union of two sets can result in a more comprehensive attack techniques view about the same attack campaign;
- Case 3: ($(ST_i \cap ST_j \neq \emptyset) \wedge (ST_i \subset ST_j \wedge ST_j \subset ST_i)$), in which instance one of two sets does not provide valuable information;
- Case 4: ($(ST_i \cap ST_j \neq \emptyset) \wedge (ST_i = ST_j)$), ST_i and ST_j are equal.

We employ the *Jaccard index* (Jaccard, 1912) described as $J(ST_i, ST_j) = \frac{|ST_i \cap ST_j|}{|ST_i \cup ST_j|}$ to calculate the similarity of two sets. $J(ST_i, ST_j) \in [0, 1]$. The higher $J(ST_i, ST_j)$, the more similar ST_i and ST_j are. The way to judge the above cases can be formalized as:

$$J_c(ST_i, ST_j) = \begin{cases} \text{Case 1} & J(ST_i, ST_j) = 0 \\ \text{Case 4} & J(ST_i, ST_j) = 1 \\ \text{Case 2, Case 3} & \text{otherwise} \end{cases} \quad (10)$$

To identify the Case 2 and Case 3, we define the *contained similarity* metric as Eq. (11). When C_s is 1, it means either one set is contained in another one or both sets are identical (Case 3). Conversely, ST_i and ST_j fall into Case 2.

$$C_s(ST_i, ST_j) = \frac{|ST_i \cap ST_j|}{\min(|ST_i|, |ST_j|)} \quad (11)$$

Based on the above analysis, we describe the process of attack technique fusion in Algorithm 2.

4. Evaluation

In this section, we aim to test and evaluate the whole performance of TIMFuser.

4.1. Dataset

To train a relevant long text identifier, we manually label the class for crawled long CTI reports, consisting of 516 attack campaign *relevant* reports or 223 *irrelevant* reports. For the relevant sentence classifier, we annotate a balanced dataset of 14204 sentences sampled from various sources, categorized into two classes: *attack behavior relevant* and *attack behavior irrelevant*. In total, 10748 sentences are labeled as relevant, and

Algorithm 2: Attack technique fusion

Input: Attack techniques and attack behaviors list
 $ST = \{ST_1, ST_2, \dots, ST_m\}$ and $SU = \{SU_1, SU_2, \dots, SU_m\}$, disclosed by m sources.

Output: Enriched attack techniques $\langle TF, UF \rangle$.

```

1 Function GetInd( $L, R, item$ ): ▷ Get index of item
2    $x \leftarrow L.index(item)$ ;
3    $y \leftarrow R.index(item)$ ;
4   return  $x, y$ 
5 End Function
6  $TF \leftarrow \emptyset, UF \leftarrow \emptyset$ ; // Initialize empty sets
7 for  $i = 1$  to  $m$  do
8    $case\_cur = J_c(TF, ST_i)$ ; ▷ Eq. (10)
9   if  $case\_cur$  is Case 1 then ▷ Case 1
10     $TF \leftarrow TF \cup ST_i, UF \leftarrow UF \cup SU_i$ ;
11   else if  $case\_cur$  is Case 4 then ▷ Case 4
12    continue;
13   else
14     $cs\_cur = C_s(TF, ST_i)$ ; ▷ Eq. (11)
15    if  $cs\_cur == 1$  then ▷ Case 3
16      if  $TF \supset ST_i$  then
17        for  $item$  in  $ST_i$  do
18           $x, y \leftarrow GetInd(ST_i, TF, item)$ ;
19           $UF[y] \leftarrow SU_i[x] \cup UF[y]$ ;
20        end
21      else
22        for  $item$  in  $TF$  do
23           $x, y \leftarrow GetInd(ST_i, TF, item)$ ;
24           $SU_i[x] \leftarrow SU_i[x] \cup UF[y]$ ;
25        end
26         $UF \leftarrow SU_i, TF \leftarrow ST_i$ ;
27      end
28    else ▷ Case 2
29       $\Gamma \leftarrow TF \cap ST_i, \Psi \leftarrow UF \cap SU_i$ ;
30      for  $item$  in  $\Gamma$  do
31         $x, y \leftarrow GetInd(ST_i, TF, item)$ ;
32         $SU_i[x], UF[y] \leftarrow SU_i[x] \cup UF[y]$ ;
33      end
34       $UF \leftarrow SU_i \cup UF, TF \leftarrow ST_i \cup TF$ 
35    end
36  end
37 end

```

5179 are annotated as irrelevant. Additionally, we create gold clusters to assess the performance of attack behaviors fusion. We label a total of 1221 entity clusters. To explore the TIMFuser’s analytical capability in the real-world attack campaign, we manually label the ground truth of the attack techniques and attack behaviors for the 11 CTI reports collected. Table 5 provides details about the dataset, including entities, dependencies, and techniques.

4.2. Evaluation metrics

The relevance identification task is considered as a binary decision problem, with outcomes categorized as positive or negative. We define the confusion matrix as illustrated in Table 6. T_p (True positive) represents the count of accurately classified samples with threat relevance. Conversely, F_p (False positive) signifies the misclassification of samples with threat irrelevance as relevant. Similarly, T_n (True negative) corresponds to the number of samples correctly classified as threat irrelevance, and F_n (False negative) denotes the misclassification of samples with threat relevance as irrelevant. Based on the confusion

Table 5
Statistical information of CTI reports from the real-world attack campaign.

CTI Reports	CTI Sources	#Entities	#Dependencies	#Techniques
Highly Evasive Attacker Leverages SolarWinds Supply Chain to Compromise Multiple Global Victims With SUNBURST Backdoor (CR1)	FireEye	52	7	17
Dark Halo Leverages SolarWinds Compromise to Breach Organizations (CR2)	Voletixity	23	4	22
Early Bird Catches the Wormhole: Observations from the StellarParticle Campaign (CR3)	CrowdStrike	50	5	23
IRON RITUAL (CR4)	MSTIC	8	2	4
NOBELIUM targeting delegated administrative privileges to facilitate broader attacks (CR5)	SecureWorks	6	3	1
GoldMax, GoldFinder, and Sibot: Analyzing NOBELIUM's layered persistence (CR6)	MSTIC	63	7	5
New SUNSHUTTLE Second-Stage Backdoor Uncovered Targeting U.S.-Based Entity (CR7)	Mandiant	12	6	1
Further TTPs associated with SVR cyber actors (CR8)	NCSC	15	3	7
SUNSPOT: An Implant in the Build Process (CR9)	CrowdStrike	15	6	9
SUNBURST Additional Technical Details (CR10)	Mandiant	25	7	8
SUNBURST, TEARDROP and the NetSec New Normal (CR11)	CheckPoint	6	2	2

Table 6
Confusion matrix.

Actual Labels/Predicted Labels	Relevance	Irrelevance
Relevance	T_p	F_n
Irrelevance	F_p	T_n

matrix, we use the following metrics:

$$Precision = T_p / (T_p + F_p) \quad (12)$$

$$Recall = T_p / (T_p + F_n) \quad (13)$$

$$F1 - score = \frac{2 \times Precision \times Recall}{Precision + Recall} \quad (14)$$

To evaluate the effect of attack behavior fusion in attack graph extraction, we manually construct gold entity clusters. Note that we do not evaluate actions fusion performance because all our actions are uniform. We use the evaluation metrics of [Sarhan and Spruit \(2021\)](#), i.e. *macro*, *micro*, and *pairwise*. We consider the *precision*, *recall*, and *F1-score* of these metrics. *F1-score* is defined in Eq. (14). The formalization is as follows. Let C represent the clusters produced by our fusion model, and G be the gold standard clusters.

- **Macro:** Macro precision (P_{macro}) denotes the percentage of generated clusters linked to gold standard clusters. Macro recall (R_{macro}) is the inverse of P_{macro} .

$$P_{macro}(C, G) = \frac{|c \in C : \exists g \in G : g \supseteq c|}{|C|} \quad (15)$$

$$R_{macro}(C, G) = P_{macro}(G, C) \quad (16)$$

- **Micro:** Micro precision (P_{micro}) measures the purity of the clusters C under the assumption that the most frequent gold entity among the mentions in a cluster is the correct entity. Micro recall (R_{micro}) is the inverse of P_{micro} .

$$P_{micro}(C, G) = \frac{1}{N} \sum_{c \in C} \max_{g \in G} |c \cap g| \quad (17)$$

$$R_{micro}(C, G) = P_{micro}(G, C) \quad (18)$$

- **Pairwise:** Pairwise precision (P_{pair}) quantifies the ratio of the number of hits in C to the total possible pairs in C . Pairwise recall (R_{pair}) is computed as the ratio of hits in C to all possible pairs

in G . A *hit* within cluster C indicates that two mentions refer to the same gold entity.

$$P_{pair}(C, G) = \frac{\sum_{c \in C} |\{(v, v') \in g, \exists g \in G, \forall (v, v') \in c\}|}{\sum_{c \in C} |c| C_2} \quad (19)$$

$$R_{pair}(C, G) = \frac{\sum_{c \in C} |\{(v, v') \in g, \exists g \in G, \forall (v, v') \in c\}|}{\sum_{g \in G} |g| C_2} \quad (20)$$

4.3. Evaluation setup

To assess the effectiveness of TIMFuser, we conduct a set of experiments. We compare the methods of each subtask in TIMFuser with existing mainstream models (See Section 4.4), setting parameters not mentioned to their default values.

4.4. Results and analysis

In order to explain the ability of TIMFuser more clearly and intuitively, we have designed **five questions** and provide in-depth answers. Especially, we also give the experimental analysis and ablation evaluations.

Q1: How accurate is TIMFuser in identifying relevance from noisy multi-sources data?

We split the training set and the test set with a ratio of 3:1 for the relevance recognition task. For the relevant long text classification subtask, we use SOTA long text classification models as baselines to prove our performance, including: (1) **BERT (document truncation)** ([Devlin et al., 2018](#)) truncates the long text to the first 512 tokens and processes the truncated content; (2) **BERT+TextRank (key sentence selection)** ranks and select key sentences (up to 512 tokens); (3) **BERT+Random (key sentence selection)** ([Park et al., 2022](#)) selects random sentences up to 512 tokens to augment the first 512 tokens; (4) **ToBERT (segment representations)** ([Pappagari et al., 2019](#)) splits the input long text into shorter segments of 200 tokens to obtain a representation for each of them.

Fig. 9 shows the results of relevant long-text classification. Our method achieves the best performance with an improved F1-score of 6.31%. This demonstrates that TIMFuser can accurately identify relevant information even from noisy multi-source data. This further proves the effectiveness of our method. Since other baseline models processing the truncated text or segmented representations may lose key information for classification, they have poor performance. We choose the best method to train our model. During training, we use a mini-batch size of 4 and train the model for 50 epochs. Following experimental comparisons, we ultimately select Cross Entropy as the

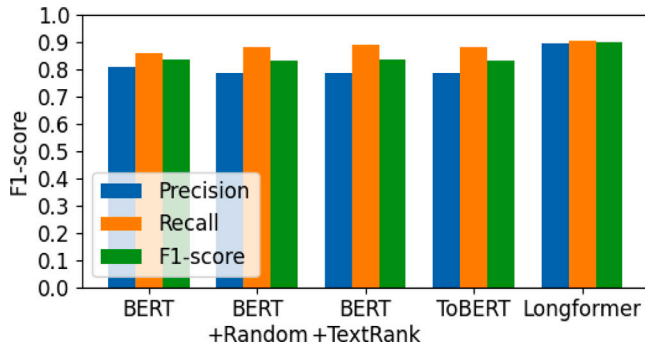


Fig. 9. Results for the relevant long text classification subtask.

Table 7
Comparison results of different embedding method.

Models	Precision	Recall	F1-score
Word2vec	76.12%	85.94%	80.73%
Glove	77.82%	86.47%	81.92%
BERT-base-cased	72.78%	94.87%	82.37%
BERT-base-uncased	97.67%	94.23%	92.04%
BERT-large-uncased	82.95%	94.70%	88.44%
RoBERTa-base	98.66%	99.66%	99.16%
RoBERTa-large	98.88%	99.66%	99.27%

loss function, and Adam (Kingma and Ba, 2014) as the optimizer with a learning rate of $3e-5$.

Due to the superior results of the transformer models in the classification task (see Fig. 9), we finetune a series of large models including variants (BERT-base-cased, BERT-base-uncased, BERT-large-uncased) of BERT (Devlin et al., 2018), and variants (RoBERTa-base, RoBERTa-large) of RoBERTa (Liu et al., 2021) as encoders for the relevant sentence classification subtask. We compare them with classical word embedding models such as Word2vec (Wu et al., 2018), Glove (Wu et al., 2018), etc. Comparative results in Table 7 demonstrate that RoBERTa-large achieves the best performance. This superior performance is mainly attributed to the dynamic mask mechanism of RoBERTa and the vast corpus at the scale of billions, which may exhibit superior generalization compared to other methods in the specific domain. We choose RoBERTa-large as our effective encoder to train our model. During training, the sequence length is set to 150 and the learning rate is set to $5e-5$. We still use Adam as the optimizer. The experimental results (see Table 8) are compared with existing SOTA machine learning models such as support vector machines (SVM) (Cortes and Vapnik, 1995) and deep learning methods (MLP (Pouyanfar et al., 2018), fastText (Joulin et al., 2017), TextCNN (Kim, 2014), BiLSTM (Li et al., 2019b) stacked with RoBERTa-large embedding layer). From the comparison of different measures, our model outperforms other learning methods, even though machine learning algorithms achieve good precision. This indicates that TIMFuser can reduce the redundancy in CTI reports and capture critical attack details effectively. This success is greatly attributed to the powerful learning and representation capabilities of large language models. In the ablation assessment by comparing with BiLSTM (Li et al., 2019b), we find that the attention mechanism can improve the F1-score by 2.31%.

By removing CTI reports unrelated to attack campaigns and sentences that are not strictly relevant to attack behaviors, TIMFuser ensures the performance of subsequent tasks.

Q2: Whether the attack behavior fusion module used to extract attack graph is adequate and the best?

We use glove to initialize embeddings for attack behaviors, as shown in Algorithm 1. Table 9 presents the comparative results of attack behaviors fusion with SOTA. Our method performs best with the F1-scores of macro, micro, and pairwise improved by 1.9%, 2.74%,

Table 8
Results for the relevant sentence recognition subtask.

Models	Precision	Recall	F1-score
SVM (Cortes and Vapnik, 1995)	87.65%	84.04%	85.80%
MLP (Pouyanfar et al., 2018)	80.29%	88.89%	84.37%
fastText (Joulin et al., 2017)	78.58%	72.81%	75.59%
TextCNN (Kim, 2014)	90.22%	98.31%	94.09%
BiLSTM (Li et al., 2019b)	94.20%	99.89%	96.96%
Ours	98.88%	99.66%	99.27%

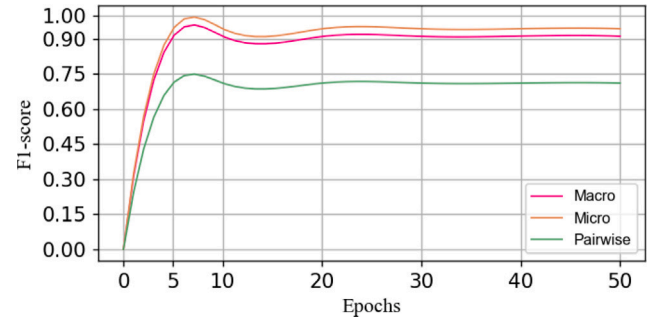


Fig. 10. F1-scores of TIMFuser for different epochs.

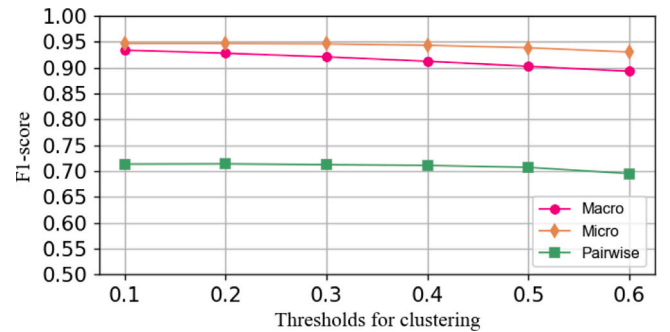


Fig. 11. F1-scores of TIMFuser for different clustering thresholds.

5.49%, respectively. This is because we combine the side information and learned embeddings (semantic and structural information). In contrast, Li et al. (2022) and Guo et al. (2023), relying on the edit distance method, consider only character features, which are less effective than TIMFuser. Sarhan and Spruit (2021) use a clustering method for CTI fusion, but embeddings from other tasks cannot capture the equivalent entities. Moreover, Sarhan and Spruit (2021) do not utilize side information. We replicate the approach of Sarhan and Spruit (2021), directly using embeddings from the relevant sentence recognition subtask, then performing clustering to identify equivalent entities and actions. Additionally, We replicate the fusing methods of Li et al. (2022) and Guo et al. (2023), separately adjusting the distance thresholds to 2 and 5 (differing from the original work) and other details to achieve the optimal results. These findings highlight the significant contributions of our side information and learned embeddings.

Figs. 10 and 11 depict the trends of different metrics with varying epochs and thresholds of HAC clustering. During the training process, as the number of training epochs increases, F1 gradually increases and eventually stabilizes. The F1-scores of macro, micro, and pairwise reach 93.36%, 94.66%, and 71.36%, respectively. It is noteworthy that pairwise F1-score is relatively low, indicating that not all mentions of entities in *C* refer to the same gold entity. The distance threshold is changed from [0.1, 0.6]. When the threshold is 0.1, TIMFuser achieves the best results.

To explore the effect of side information, we conduct ablation experiments shown in Fig. 12 by using different variants of TIMFuser.

Table 9
Comparison results for attack behavior fusion.

Model	Macro			Micro			Pairwise		
	Precision	Recall	F1-score	Precision	Recall	F1-score	Precision	Recall	F1-score
Sarhan and Spruit (2021)	86.45%	87.38%	86.91%	76.74%	91.82%	83.61%	55.70%	80.57%	65.87%
Li et al. (2022)	95.61%	87.66%	91.46%	89.45%	94.54%	91.92%	43.01%	92.73%	58.76%
Guo et al. (2023)	65.22%	88.00%	74.92%	73.64%	95.33%	83.09%	46.32%	81.60%	59.09%
TIMFuser (Ours)	98.29%	88.91%	93.36%	94.53%	94.79%	94.66%	57.82%	93.17%	71.36%

Table 10
Comparison results for F1-scores in attack graph extraction and attack techniques recognition.

CTI reports	Entities			Dependencies			Techniques	
	EXTRACTOR	AttackKG	TIMFuser	EXTRACTOR	AttackKG	TIMFuser	AttackKG	TIMFuser
CR1	58.56%	54.89%	91.80%	30.77%	72.57%	93.33%	71.62%	88.04%
CR2	58.02%	45.28%	94.11%	49.28%	76.92%	100.00%	64.24%	82.14%
CR3	43.87%	46.02%	86.24%	77.48%	77.12%	90.91%	61.35%	78.03%
CR4	50.00%	58.72%	77.78%	71.44%	63.22%	100.00%	63.32%	76.93%
CR5	67.06%	53.33%	76.92%	33.33%	59.30%	80.00%	67.49%	79.65%
CR6	35.40%	61.06%	84.27%	42.92%	71.89%	93.33%	67.57%	86.27%
CR7	54.31%	56.78%	68.09%	46.52%	90.20%	92.31%	67.72%	80.90%
CR8	48.74%	43.75%	76.47%	46.15%	62.86%	100.00%	49.69%	86.27%
CR9	53.79%	57.12%	80.00%	52.52%	90.54%	100.00%	62.44%	87.53%
CR10	45.65%	64.21%	65.31%	57.06%	64.80%	100.00%	67.80%	76.00%
CR11	62.07%	47.65%	83.33%	51.50%	57.68%	100.00%	69.43%	85.33%
Average	52.50%	53.53%	80.39%	50.82%	71.55%	95.44%	64.79%	82.46%
F1-score								

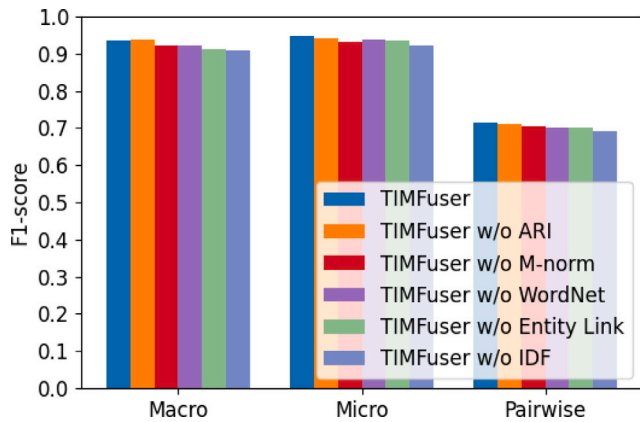


Fig. 12. Ablation experiments on side information of TIMFuser.

When any one of the five side information is removed, F1-scores of macro, micro, and pairwise are all reduced. Notably, IDF token overlap side information has the most significant effect on our model, with the micro F1-score reduced by 3.72%. This is due to the fact that threat descriptions contain many overlapping entities, such as “cardinal rat” versus “cardinal”.

In conclusion, our method benefiting from side information can improve performance. This not only reflects the effectiveness of our fusion approach but also emphasizes the practical significance of incorporating knowledge or side information into CTI. It assists in the discovery of subtle relationships between attack behaviors, ultimately providing a more accurate representation of attack behaviors.

Q3: How accurate is TIMFuser in extracting attack graph from CTI reports?

For the answer of Q3, we evaluate our method using the aforementioned 11 well-labeled CTI reports from the same attack campaign (e.g., Solarwinds). Comparing our F1-scores with SOTA methods (see Table 10) in entities and dependencies recognition, such as AttackKG (Li et al., 2022), EXTRACTOR (Satvat et al., 2021), reveals that AttackKG and EXTRACTOR have lower F1-scores in entities and dependencies identification. EXTRACTOR aggregates all non-IoC entities

of the same type into one entity and more details of the attack are missing (e.g., structural information), which results in a lower accuracy rate. On the other hand, AttackKG fuses entities and constructs attack graphs based on character features, thereby resulting in a high false positives rate. For instance, “APT28” and “APT29” are fused together due to their edit distance being less than a threshold, even though they belong to different attack groups. In contrast, TIMFuser improves the average F1-scores by 26.86% and 23.89%, respectively, compared to other baselines.

These significant improvements highlight the effectiveness of TIMFuser in accurately identifying and representing entities and their dependencies within the attack graphs. This is crucial for security analysts, as it minimizes false positives in real-world attack scenarios.

Q4: What is the performance of TIMFuser in identifying attack techniques from CTI reports?

To answer Q4, we evaluate the SOTA technique recognizer e.g., AttackKG (Li et al., 2022), on the 11 CTI reports. It is noteworthy that AttackKG cannot recognize the sub-technologies. Nonetheless, we consider it correct when it can identify technologies associated with the gold sub-technology. The last three columns in Table 10 summarize the technique identification results. TIMFuser significantly outperforms AttackKG in terms of the F1-score. This result makes sense as TIMFuser considers the threat context and structure information of attack behaviors. TIMFuser combines the contextual information of attack behaviors for effective identification, enhancing the accuracy of attack technique identification and further ensuring a more comprehensive threat landscape.

Q5: Whether the attack technique fusion module used to construct the complete view of attack campaign is adequate and optimal?

In our attack techniques recognition subtask, we analyze the distribution of attack techniques from different CTI sources. CrowdStrike, Volexity, FireEye offer the significant contributions, 32.32%, 22.22%, and 17.17%, respectively. To evaluate the capability of our attack techniques fusion, we design a set of tests: (T1) combines all techniques from all CTI sources directly, (T2) eliminates duplicates based on T1. Compared to T2, TIMFuser can correlate the attack techniques and generate the enriched (fused) attack techniques. We present the counts of enriched attack techniques from the above three situations in Table 11. The count of fused attack techniques generated by TIMFuser

Table 11

Tests for the attack techniques fusion. The last two rows indicate the counts of attack techniques and attack behaviors.

Fused attack techniques	Tests		
	T1	T2	TIMFuser
Techniques (<i>ST</i>)	80	61	61
Attack behaviors (<i>SU</i>)	456	421	263

is small. This result makes sense as TIMFuser benefits from duplicates and correlation. This also demonstrates the effect of attack technique fusion at the attack campaign level.

To explore the quality of fused attack techniques produced by TIMFuser, we select five representatives of fused attack techniques for analysis. Table 12 presents the descriptions of results. When analyzing our enriched attack techniques, one observation that stands out is the potential they have in establishing connections between attack behaviors of the same attack technique released at different times (columns 3 and 4 in Table 12), allowing the creation of a timeline for the evolution of the exploration of the attack campaign or an attacker. As seen in column 2, we can observe the establishment of connections between different attacks within the same attack techniques as a result of changes in targets or multiple attacks. These connections are essential to build a complete attack view for the attack campaign, enhancing the understanding of attack patterns and their evolution.

4.5. Case study

In this section, we will discuss how TIMFuser benefits downstream security tasks. Taking the Solarwinds attack campaign (mentioned in Section 2.4) as an example, TIMFuser can quickly identify related enriched attack techniques (including fused attack behaviors) from the massive CTI reports involved in this campaign. For instance, *T1195.002 - Subvert Trust Controls: Code Signing* with fused attack behaviors [$u_{14}, u_{15}, u_{16}, u_{17}, u_{21}$] (see Fig. 3) for *tactic defense evasion*, *T1098.002 - Acquire Infrastructure: Domains* with fused attack behaviors for *privilege escalation*. Once we obtain the comprehensive correlation information of attack techniques and attack behaviors, we can construct a complete attack graph denoted as *provenance graph*. Based on this provenance graph, we match it with a *query graph* constructed from system audit logs for *cyber threat hunting*. Wei et al. (2021) demonstrate that incomplete query graphs and provenance graphs are prone to cause higher false positives during cyber threat hunting. Therefore, to some extent, our complete attack graph can effectively reduce the false positive rate in threat hunting. We will verify this in our future work.

In summary, TIMFuser fuses the technical-level and attack campaign-level CTIs to facilitate the construction of a comprehensive view (see Fig. 2) of the attack campaign.

5. Discussion

In this paper, we present a novel multi-granular CTI fusion framework, TIMFuser, which parses CTI reports and fuses CTIs from multiple sources at both the attack technique and attack campaign level with a high accuracy. However, there are some aspects for further improvement.

Real-time Processing: TIMFuser utilizes the pipelined structure, which hinders the immediate generation of conclusive results, potentially limiting its application in specific scenarios. This constraint can be alleviated in the future through the development of an end-to-end framework.

Attack Scenarios Evaluation: The current evaluation of TIMFuser primarily relies on a restricted set of attack scenarios, potentially limiting its scalability in diverse attack scenarios. However, since TIMFuser is modular in design, users can adjust the selection of data sources, set the analysis granularity, and modify the parameters of the fusion

algorithms as needed to ensure the system can adapt to diverse attack analysis tasks. Nonetheless, a comprehensive evaluation of TIMFuser across a broader range of attack scenarios is crucial to ensuring its effectiveness in addressing various cyber threats.

Large-scale multi-source data processing: TIMFuser faces challenges when processing large-scale multi-source data. Considering that processing large-scale data may increase the time required for TIMFuser to collect and analyze data, this not only slows down the entire process but may also affect the timeliness of CTI (Ren et al., 2022). Maximizing the timeliness of TIMFuser may result in collecting insufficient data. If the time window for collecting data is too large, data would be sufficient, but the influence of TIMFuser on providing decision assistance for security analysts would be small. To strike a balance between the timeliness of CTI and the quality of CTI fusion, TIMFuser relies on a seed URL list related to the specific attack campaign (usually manually curated), covering certain CTI sources, and employs a breadth-first-search (BFS) algorithm to gather data. This approach ensures the abundance of data sources while reducing the occurrence of noisy data. Simultaneously, it maintains the timeliness of the CTI, thereby improving the overall performance of TIMFuser. Furthermore, as the volume of data increases, TIMFuser can enhance its ability to process large-scale data by incorporating technologies such as distributed architectures.

6. Related work

In this section, we mainly review previous work related to this paper from two aspects: **CTI extraction** and **CTI fusion**.

6.1. Cyber threat intelligence extraction

Existing open-source systems for threat intelligence extracting primarily focus on low-level IoCs (Gao et al., 2020; Liao et al., 2016; Liu et al., 2022; Zhao et al., 2020; Zhu and Dumitras, 2018). While IoCs are valuable for capturing localized and fragmented information about an attack, they suffer from being low-level and poorly correlated. Consequently, they lack the capability to unveil the complete attack scenario and provide limited insights into how the attack is conducted. In contrast, attack behaviors play a pivotal role in revealing complete, multi-step attack scenarios and developing more robust defense strategies. Currently, methods for attack behavior extraction can be classified into ontology model-based techniques, common machine learning-based methods, and deep learning-based approaches.

Ontology model-based Techniques. Husari et al. (2017) propose an approach called TTPDrill, which utilizes semantic dependencies and ontology databases to extract actions and map them to various attack patterns. Nevertheless, TTPDrill neglects certain attack behaviors when addressing subordinate clause structures and compound sentences. Additionally, the ontology model cannot cover all undefined attack behaviors.

Machine Learning-based Methods. Zhu and Dumitras (2016) propose an end-to-end automated feature engineering approach. This method aims to identify malware-related behaviors and map them to specific features, ultimately generating a semantic feature network. Gao et al. (2021) employ an unsupervised, lightweight natural language pipeline processing technique. Specifically, they utilize regular expressions for extracting threat entities, followed by dependency parsing to identify relationships among these entities, thereby forming attack behaviors. Husari et al. (2018) develop ActionMiner to extract low-level attack behaviors by utilizing information entropy and mutual information. They finally link these attack behaviors to TTPs and the attack chain. However, ActionMiner relies on syntactic analysis and lacks the subjects of attack behaviors, making it challenging to ensure content accuracy. Zhang et al. (2021) present the EX-Action framework, which enhances the accuracy of attack behavior recognition through a weighted ensemble learning algorithm incorporating various types of features. Compared to ActionMiner and TTPDrill, EX-Action can

Table 12
Analysis of fused attack techniques.

Fused attack techniques	#Related attack techniques	Earliest attack behavior date	Latest attack behavior date	Span ↓
T1560.001	3	2020/12/14	2022/01/27	409 days
T1133	2	2021/03/04	2022/01/27	329 days
T1098.002	2	2020/12/14	2021/10/25	315 days
T1078	3	2020/12/13	2021/05/07	145 days
T1053.005	3	2020/12/13	2021/01/11	29 days

not only identify and extract attack behaviors in complex sentence structures and semantic relationships but also can recognize threat entity associations with undefined relationships. Additionally, it performs well in terms of the number of attack behaviors extracted and the ability to maintain information in complex sentence structures.

Deep Learning-based Approaches. Satvat et al. (2021) introduce the EXTRACTOR system, designed for extracting attack behaviors from open-source CTI reports and conducting cyber threat hunting. The system employs a deep learning model based on semantic role labeling to analyze complex sentence structures and infers facts like “*who did what to whom*” “*when*” and “*where*”. Additionally, Gao et al. (2022) enhance their earlier work (Gao et al., 2021) by adopting a deep learning-based approach for extracting attack behaviors. Nevertheless, the use of pipelined processing may introduce an error propagation issue, leading to a higher occurrence of false positives in the extracted attack behaviors.

6.2. Cyber threat intelligence fusion

The effective utilization of CTI fundamentally relies on multi-source heterogeneous fusion, aiming to correlate and fuse intelligence from diverse sources using machine learning and other methods. This process results in high-quality CTIs characterized by timeliness, accuracy, and completeness. Recent research in CTI fusion has concentrated on three phases: consistency analysis, duplicate removal, and entity fusion.

Consistency Analysis. Ontology serves as the semantic foundation for communication and connectivity among different entities in the same domain (Studer et al., 1998). Ontology construction is a crucial prerequisite for consistency analysis. Zhao et al. (2017) design a cyber-security ontology model that maps CTIs from different sources into a unified representation. Furthermore, they establish the CTI integration framework *IntelMQ* based on this ontology model and an open-source CTI collection tool. Jo et al. (2020) employ a deep learning approach to analyze the functional descriptions of malware. They find that there are syntactic and semantic inconsistencies in these functional descriptions from multiple sources. To address this issue, they develop a consistency detection system called *GapFinder*.

Duplicate Removal. Duplicate removal constitutes another essential processing step in CTI fusion. Brown et al. (2015) utilize a rapid matching algorithm to precisely identify matching records from diverse CTI sources. Subsequently, they perform merging and deduplication based on attributes, relationships, content, and other dimensions. Sun et al. (2021) initially conduct consistency analysis and duplicate removal on CTIs from multiple sources. Subsequently, they employ higher-order analysis techniques like support vector machine (SVM) for cleaning, integration, and consolidation of CTIs. Finally, the generated CTI records are normalized using Structured Threat Information Expression (STIX) and are stored in a Neo4j database.

Entity Fusion. Several studies (Modi et al., 2016; Azevedo et al., 2019) have employed clustering for CTI fusion. Modi et al. (2016) present an automated CTI fusion framework to aggregate isolated cyber events and correlate CTIs with similar content. Azevedo et al. (2019) utilize an enhanced clustering approach to aggregate similar IOCs from diverse CTI sources and generate the enriched IOCs format. Nevertheless, these methods perform CTI fusion in a coarse-grained way. Li et al. (2022) conduct the fusion of attack behaviors at the attack

technique level using Levenshtein distance. Guo et al. (2023) enhance the Levenshtein distance method to fuse attack group entities. However, these approaches only focus on character-level features during entity fusion for CTIs. In contrast, we implement a fine-grained approach that considers both structural and semantic features for fusing attack behaviors.

7. Conclusion

In this paper, we propose TIMFuser, a multi-granular fusion framework for CTIs across extensive reports, and evaluate its performance in the real-world attack campaign. Experimental results demonstrate that TIMFuser can enable security practitioners to obtain comprehensive insights into the real-world attack campaigns. This includes fused attack techniques and attack behaviors derived from the novel, enhanced, and multi-granular fuse method. For future work, we plan to leverage these fused CTIs for cyber threat detection.

CRedit authorship contribution statement

Chunyan Ma: Writing – original draft, Software, Methodology, Investigation, Conceptualization. **Zhengwei Jiang:** Writing – review & editing, Resources, Formal analysis. **Kai Zhang:** Resources, Formal analysis. **Zhiting Ling:** Resources, Formal analysis. **Jun Jiang:** Resources, Formal analysis. **Yizhe You:** Methodology, Investigation. **Peian Yang:** Writing – review & editing, Resources, Methodology, Formal analysis. **Huamin Feng:** Writing – review & editing, Methodology, Formal analysis.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgments

This research is supported by the Youth Innovation Promotion Association, CAS (No. 2020166). This work is also supported by the Program of Key Laboratory of Network Assessment Technology, the Chinese Academy of Sciences; Program of Beijing Key Laboratory of Network Security and Protection Technology.

References

- Abdeen, B., Al-Shaer, E., Singhal, A., Khan, L., Hamlen, K., 2023. Smet: Semantic mapping of cve to att&ck and its application to cybersecurity. In: IFIP Annual Conference on Data and Applications Security and Privacy. Springer, pp. 243–260.
- Alam, M.T., Bhusal, D., Park, Y., Rastogi, N., 2023. Looking beyond iocs: Automatically extracting attack patterns from external CTI. In: Proceedings of the 26th International Symposium on Research in Attacks, Intrusions and Defenses. pp. 92–108.

- Azevedo, R., Medeiros, I., Bessani, A., 2019. PURE: Generating quality threat intelligence by clustering and correlating OSINT. In: 2019 18th IEEE International Conference on Trust, Security and Privacy in Computing and Communications/13th IEEE International Conference on Big Data Science and Engineering. TrustCom/BigDataSE, IEEE, pp. 483–490.
- Beltagy, I., Peters, M.E., Cohan, A., 2020. Longformer: The long-document transformer. arXiv preprint arXiv:2004.05150.
- Bianco, D., 2013. The pyramid of pain.
- Blanda, K., 2023. APTnotes. <https://github.com/aptnotes/>.
- Bordes, A., Usunier, N., Garcia-Duran, A., Weston, J., Yakhnenko, O., 2013. Translating embeddings for modeling multi-relational data. Adv. Neural Inf. Process. Syst. 26, 1–9.
- Brown, S., Gommers, J., Serrano, O., 2015. From cyber security information sharing to threat management. In: Proceedings of the 2nd ACM Workshop on Information Sharing and Collaborative Security. pp. 43–49.
- Cash, D., Meltzer, M., Koessel, S., Adair, S., Lancaster, T., Volexity Threat Research, 2020. Dark halo leverages SolarWinds compromise to breach organizations. <https://www.volexity.com/blog/2020/12/14/dark-halo-leverages-solarwinds-compromise-to-breach-organizations/>.
- Cheng, Z., Dai, R., Wang, L., Yu, Z., Lv, Q., Wang, Y., Sun, D., 2023. Ghunter: A fast subgraph matching method for threat hunting. In: 2023 26th International Conference on Computer Supported Cooperative Work in Design. CSCWD, IEEE, pp. 1014–1019.
- Cortes, C., Vapnik, V., 1995. Support-vector networks. Mach. Learn. 20, 273–297.
- CrowdStrike, 2022. Early bird catches the wormhole: Observations from the StellarParticle campaign. <https://www.crowdstrike.com/blog/observations-from-the-stellarparticle-campaign/>.
- Dedola, G., 2020. Transparent tribe: Evolution analysis, part 1. <https://securelist.com/transparent-tribe-part-1/98127/>.
- Devlin, J., Chang, M.W., Lee, K., Toutanova, K., 2018. Bert: Pre-training of deep bidirectional transformers for language understanding. arXiv preprint arXiv:1810.04805.
- ESET, 2022. Threat report T3 2021. https://web-assets.esetstatic.com/wls/2022/02/eset_threat_report_t32021.pdf.
- Fireeye, 2023. Highly evasive attacker leverages SolarWinds supply chain to compromise multiple global victims with SUNBURST backdoor. <https://www.mandiant.com/resources/blog/evasive-attacker-leverages-solarwinds-supply-chain-compromises-with-sunburst-backdoor/>.
- Gao, Y., Li, X., Peng, H., Fang, B., Philip, S.Y., 2020. Hincti: A cyber threat intelligence modeling and identification system based on heterogeneous information network. IEEE Trans. Knowl. Data Eng. 34 (2), 708–722.
- Gao, P., Liu, X., Choi, E., Ma, S., Yang, X., Ji, Z., Zhang, Z., Song, D., 2022. ThreatKG: A threat knowledge graph for automated open-source cyber threat intelligence gathering and management. arXiv preprint arXiv:2212.10388.
- Gao, P., Shao, F., Liu, X., Xiao, X., Qin, Z., Xu, F., Mittal, P., Kulkarni, S.R., Song, D., 2021. Enabling efficient cyber threat hunting with cyber threat intelligence. In: 2021 IEEE 37th International Conference on Data Engineering. ICDE, IEEE, pp. 193–204.
- GREAT, 2023. APT trends report Q1 2023. <https://securelist.com/apt-trends-report-q1-2023/109581/>.
- Grigorescu, O., Nica, A., Dascalu, M., Rughinis, R., 2022. Cve2att&ck: Bert-based mapping of cves to mitre att&ck techniques. Algorithms 15 (9), 314.
- Guo, Y., Liu, Z., Huang, C., Wang, N., Min, H., Guo, W., Liu, J., 2023. A framework for threat intelligence extraction and fusion. Comput. Secur. 132, 103371.
- He, L., Lee, K., Lewis, M., Zettlemoyer, L., 2017. Deep semantic role labeling: What works and what's next. In: Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). pp. 473–483.
- Husari, G., Al-Shaer, E., Ahmed, M., Chu, B., Niu, X., 2017. Ttpdrill: Automatic and accurate extraction of threat actions from unstructured text of cti sources. In: Proceedings of the 33rd Annual Computer Security Applications Conference. pp. 103–115.
- Husari, G., Niu, X., Chu, B., Al-Shaer, E., 2018. Using entropy and mutual information to extract threat actions from cyber threat intelligence. In: 2018 IEEE International Conference on Intelligence and Security Informatics. ISI, IEEE, pp. 1–6.
- Ikody, A., Wagnen, G., Dulaunoy, A., Mokaddem, S., Wagner, C., 2018. Decaying indicators of compromise. arXiv preprint arXiv:1803.11052.
- Jaccard, P., 1912. The distribution of the flora in the alpine zone. 1. New Phytol. 11 (2), 37–50.
- Jo, H., Kim, J., Porras, P., Yegneswaran, V., Shin, S., 2020. GapFinder: Finding inconsistency of security information from unstructured text. IEEE Trans. Inf. Forensics Secur. 16, 86–99.
- Jo, H., Lee, Y., Shin, S., 2022. Vulcan: Automatic extraction and analysis of cyber threat intelligence from unstructured text. Comput. Secur. 120, 102763.
- Joulin, A., Grave, E., Bojanowski, P., Mikolov, T., 2017. Bag of tricks for efficient text classification. In: Proceedings of the 15th Conference of the European Chapter of the Association for Computational Linguistics: Volume 2, Short Papers. pp. 427–431.
- Kim, Y., 2014. Convolutional neural networks for sentence classification. In: Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing. EMNLP, pp. 1746–1751.
- Kingma, D.P., Ba, J., 2014. Adam: A method for stochastic optimization. arXiv preprint arXiv:1412.6980.
- Kobren, A., Monath, N., Krishnamurthy, A., McCallum, A., 2017. A hierarchical algorithm for extreme clustering. In: Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. pp. 255–264.
- Li, V.G., Dunn, M., Pearce, P., McCoy, D., Voelker, G.M., Savage, S., 2019a. Reading the tea leaves: A comparative analysis of threat intelligence. In: 28th USENIX Security Symposium. USENIX Security 19, pp. 851–867.
- Li, W., Gao, S., Zhou, H., Huang, Z., Zhang, K., Li, W., 2019b. The automatic text classification method based on bert and feature union. In: 2019 IEEE 25th International Conference on Parallel and Distributed Systems. ICPADS, IEEE, pp. 774–777.
- Li, Z., Zeng, J., Chen, Y., Liang, Z., 2022. AttackKG: Constructing technique knowledge graph from cyber threat intelligence reports. In: European Symposium on Research in Computer Security. Springer, pp. 589–609.
- Liao, X., Yuan, K., Wang, X., Li, Z., Xing, L., Beyah, R., 2016. Acing the ioc game: Toward automatic discovery and analysis of open-source cyber threat intelligence. In: Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security. pp. 755–766.
- Liu, Z., Lin, W., Shi, Y., Zhao, J., 2021. A robustly optimized BERT pre-training approach with post-training. In: China National Conference on Chinese Computational Linguistics. Springer, pp. 471–484.
- Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., Levy, O., Lewis, M., Zettlemoyer, L., Stoyanov, V., 2019. Roberta: A robustly optimized bert pretraining approach. arXiv preprint arXiv:1907.11692.
- Liu, J., Yan, J., Jiang, J., He, Y., Wang, X., Jiang, Z., Yang, P., Li, N., 2022. TriCTI: an actionable cyber threat intelligence discovery system via trigger-enhanced neural network. Cybersecurity 5 (1), 8.
- Milajerdi, S.M., Eshete, B., Gjomoemo, R., Venkatakrishnan, V., 2019. Piroet: Aligning attack behavior with kernel audit records for cyber threat hunting. In: Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security. pp. 1795–1812.
- Miller, G.A., 1995. WordNet: a lexical database for english. Commun. ACM 38 (11), 39–41.
- MITRE, 2023. Data model. https://car.mitre.org/data_model/.
- Modi, A., Sun, Z., Panwar, A., Khairnar, T., Zhao, Z., Doupé, A., Ahn, G.J., Black, P., 2016. Towards automated threat intelligence fusion. In: 2016 IEEE 2nd International Conference on Collaboration and Internet Computing. CIC, IEEE, pp. 408–416.
- MSTIC, 2020. Analyzing solorigate, the compromised DLL file that started a sophisticated cyberattack, and how Microsoft Defender helps protect customers. <https://www.microsoft.com/en-us/security/blog/2020/12/18/analyzing-solorigate-the-compromised-dll-file-that-started-a-sophisticated-cyberattack-and-how-microsoft-defender-helps-protect/>.
- Muthukadan, B., 2023. Selenium with Python. <https://selenium-python.readthedocs.io/>.
- Narayanan, A., Chandramohan, M., Venkatesan, R., Chen, L., Liu, Y., Jaiswal, S., 2017. Graph2vec: Learning distributed representations of graphs. arXiv preprint arXiv:1707.05005.
- Nextron Systems, 2023a. Loki - Simple IOC and YARA scanner. <https://www.nextron-systems.com/loki/>.
- Nextron Systems, 2023b. THOR LITE, free IOC and YARA scanner. <https://www.nextron-systems.com/thor-lite/>.
- Palmer, M., Gildea, D., Kingsbury, P., 2005. The proposition bank: An annotated corpus of semantic roles. Comput. Linguist. 31 (1), 71–106.
- Pappagari, R., Zelasko, P., Villalba, J., Carmiel, Y., Dehak, N., 2019. Hierarchical transformers for long document classification. In: 2019 IEEE Automatic Speech Recognition and Understanding Workshop. ASRU, IEEE, pp. 838–844.
- Park, H.H., Vyas, Y., Shah, K., 2022. Efficient classification of long documents using transformers. arXiv preprint arXiv:2203.11258.
- Piplai, A., Mittal, S., Joshi, A., Finin, T., Holt, J., Zak, R., 2020. Creating cyber-security knowledge graphs from malware after action reports. IEEE Access 8, 211691–211703.
- Pouyanfar, S., Sadiq, S., Yan, Y., Tian, H., Tao, Y., Reyes, M.P., Shyu, M.L., Chen, S.C., Iyengar, S.S., 2018. A survey on deep learning: Algorithms, techniques, and applications. ACM Comput. Surv. 51 (5), 1–36.
- Rani, N., Saha, B., Maurya, V., Shukla, S.K., 2023. Ttphunter: Automated extraction of actionable intelligence as TTPs from narrative threat reports. In: Proceedings of the 2023 Australasian Computer Science Week. pp. 126–134.
- Ren, Y., Xiao, Y., Zhou, Y., Zhang, Z., Tian, Z., 2022. Csk4apt: A cybersecurity knowledge graph for advanced persistent threat organization attribution. IEEE Trans. Knowl. Data Eng. 35 (6), 5695–5709.
- Richardson, L., 2023. BeautifulSoup. <https://www.crummy.com/software/BeautifulSoup/>.
- Sarhan, I., Spruit, M., 2021. Open-cykg: An open cyber threat intelligence knowledge graph. Knowl.-Based Syst. 233, 107524.
- Satvat, K., Gjomoemo, R., Venkatakrishnan, V., 2021. Extractor: Extracting attack behavior from threat reports. In: 2021 IEEE European Symposium on Security and Privacy. EuroS&P, IEEE, pp. 598–615.
- Stojanović, B., Hofer-Schmitz, K., Kleb, U., 2020. APT datasets and attack modeling for automated detection methods: A review. Comput. Secur. 92, 101734.

- Studer, R., Benjamins, V.R., Fensel, D., 1998. Knowledge engineering: Principles and methods. *Data Knowl. Eng.* 25 (1–2), 161–197.
- Sun, T., Yang, P., Li, M., Liao, S., 2021. An automatic generation approach of the cyber threat intelligence records based on multi-source information fusion. *Future Internet* 13 (2), 40.
- Unit42, 2020. SolarStorm supply chain attack timeline. <https://unit42.paloaltonetworks.com/solarstorm-supply-chain-attack-timeline/>.
- Vashishth, S., Jain, P., Talukdar, P., 2018. Cesi: Canonicalizing open knowledge bases using embeddings and side information. In: *Proceedings of the 2018 World Wide Web Conference*. pp. 1317–1327.
- Wei, R., Cai, L., Zhao, L., Yu, A., Meng, D., 2021. Deephunter: A graph neural network based approach for robust cyber threat hunting. In: *Security and Privacy in Communication Networks: 17th EAI International Conference, SecureComm 2021, Virtual Event, September 6–9, 2021, Proceedings, Part I* 17. Springer, pp. 3–24.
- Wu, L., Yen, I.E.H., Xu, K., Xu, F., Balakrishnan, A., Chen, P.Y., Ravikumar, P., Witbrock, M.J., 2018. Word mover's embedding: From Word2Vec to document embedding. In: *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*. pp. 4524–4534.
- You, Y., Jiang, J., Jiang, Z., Yang, P., Liu, B., Feng, H., Wang, X., Li, N., 2022. TIM: threat context-enhanced TTP intelligence mining on unstructured threat data. *Cybersecurity* 5 (1), 1–17.
- Zhang, H., Shen, G., Guo, C., Cui, Y., Jiang, C., 2021. Ex-action: Automatically extracting threat actions from cyber threat intelligence report based on multimodal learning. *Secur. Commun. Netw.* 2021 (1), 1–12.
- Zhao, Y., Lang, B., Liu, M., 2017. Ontology-based unified model for heterogeneous threat intelligence integration and sharing. In: *2017 11th IEEE International Conference on Anti-Counterfeiting, Security, and Identification. ASID, IEEE*, pp. 11–15.
- Zhao, J., Yan, Q., Li, J., Shao, M., He, Z., Li, B., 2020. TIMiner: Automatically extracting and analyzing categorized cyber threat intelligence from social data. *Comput. Secur.* 95, 101867.
- Zhu, Z., Dumitras, T., 2016. Featuresmith: Automatically engineering features for malware detection by mining the security literature. In: *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*. pp. 767–778.
- Zhu, Z., Dumitras, T., 2018. Chainsmith: Automatically learning the semantics of malicious campaigns by mining threat intelligence reports. In: *2018 IEEE European Symposium on Security and Privacy. EuroS&P, IEEE*, pp. 458–472.
- Chunyan Ma** received her B.S. degree from Northwest Normal University in 2020. She is presently pursuing the Ph.D. degree at the School of Cyber Security, University of Chinese Academy of Sciences. Her current research interests include cyber threat intelligence and cyber threat detection.
- Zhengwei Jiang** received his Ph.D. degree from the University of Chinese Academy of Sciences in 2014. He is currently a Senior Engineer at the Institute of Information Engineering, Chinese Academy of Sciences, and a professor at the School of Cyber Security, University of Chinese Academy of Sciences. His research interests include cyber threat intelligence and cyber threat detection.
- Kai Zhang** received his Ph.D. degree from the School of Information and Communication Engineering, Beijing University of Posts and Telecommunications in 2020. His research interests include network security and machine learning.
- Zhitong Ling** received her M.S. degree in Beijing Electronic Science and Technology Institute in 2019. Her research interests include cyber space security and cyber threat intelligence.
- Jun Jiang** received his M.S. degree in Electronic Science and Technology from Beijing Jiaotong University in 2016. He is currently an Associate Senior Engineer at the Institute of Information Engineering, Chinese Academy of Sciences. His research interests include cyber space security and cyber threat intelligence.
- Yizhe You** received his Ph.D. degree in University of Chinese Academy of Sciences. He current works in China Mobile Security Information Center.
- Peian Yang** received his Ph.D. degree in the University of Chinese Academy of Sciences in 2018. He is currently an Associate Senior Engineer at the Institute of Information Engineering, Chinese Academy of Sciences. His research interests include cyber threat intelligence and cyber threat attribution.
- Huamin Feng** received his Ph.D. degree in 1984. He is currently a professor at the Beijing Electronic Science and Technology Institution and a visiting professor at the School of Cybersecurity, University of Chinese Academy of Sciences. His research interests include cyber security situational awareness and Internet Measurement.